Paulo Gonçalves Magalhães

Comparação entre Estratégias de Detecção de Intersecção entre Corpos Rígidos Convexos

São Paulo – Brasil

2020

Paulo Gonçalves Magalhães

Comparação entre Estratégias de Detecção de Intersecção entre Corpos Rígidos Convexos

Monografia apresentada na disciplina Trabalho de Conclusão de Curso, como parte dos requisitos para obtenção do título de Bacharel em Ciência da Computação.

Centro Universitário Senac - Santo Amaro

Bacharelado em Ciência da Computação

Orientador: Professor Me. Adilson Konrad

São Paulo – Brasil

2020

Agradecimentos

Gostaria de agradecer e dedicar este trabalho às seguintes pessoas:

A Carolina Akemi Sepulveda Matsuo, minha namorada, por me impulsinar a ser o melhor que posso, por acreditar na minha capacidade, pela parceria, cumplicidade, carinho, amor, generosidade e incentivo.

A Danilo Augusto Mative, meu querido colega e irmão de graduação, pela convivência, troca, experiência, amizade, apoio, e tudo o que fizemos e vamos continuar fazendo juntos.

A Alexandre Savelli Bencz, ex-colega de Ciência da Computação, querido amigo e irmão, companheiro, mentor, por tudo o que me ensinou e por me instigar a progredir na profissão.

Ao meu orientador, Prof. M.e Adilson Konrad, pelo impecável exemplo em pedagogia, pelo humanismo, compreensão, mentoria, paciência, orientação e amizade.

Ao coordenador do curso de Ciência da Computação, Prof. Thiago Ribeiro Claro, pela seriedade e confiança, e por sua abertura, diálogo e atenção com os alunos da graduação.

Aos professores e ex-professores do Centro Universitário Senac Santo Amaro, em especial, a Danielle Mingatos, Daniel Azeredo, Eduardo Takeo Ueda, e Renato Ramalho, verdadeiros exemplos pessoais como acadêmicos, intelectuais e pedagogos.

Aos meus queridos colegas de graduação, por tornar nosso curso mais interessante, memorável, colorido e desafiador.

Às minhas irmãs Ana, Lília e Júlia, minhas grandes referências em ética, maturidade, seriedade, profissionalismo e intelectualidade.

Por fim, e especialmente, agradeço a meus queridos pais, Aderbal e Eva Magalhães, as pessoas mais generosas, corretas e afetuosas que conheço, como seres humanos e como acadêmicos. Meu norte nessa vida.

Resumo

A Física Computacional é utilizada para criar simulações em contextos como, entre outros, a Engenharia, o Design, jogos digitais, animação e a indústria cinematográfica. Apesar dos desafios envolvidos na implementação de simulações físicas na Computação, existem ferramentas gratuitas e proprietárias dedicadas a esta finalidade utilizadas profissionalmente, que continuam a se aprimorar até o presente.

As técnicas de intersecção entre corpos na computação, em particular, são revisitadas, incrementadas e repensadas continuamente, com o intuito de se tornarem mais performáticas e precisas, especialmente em simulações interativas, calculadas em tempo real.

Este trabalho propõe uma comparação de duas técnicas de detecção de intersecção entre corpos rígidos convexos, apresentando os conceitos de Geometria Analítica e Computacional empregados em cada algoritmo, e desenvolver uma biblioteca de intersecção que utilize uma destas técnicas, observando a análise feita e bibliografia coletada sobre cada método. Uma vez implementada, a biblioteca é testada em uma simulação para validação dos resultados e coleta de dados de performance de suas funções.

Palavras-chaves: Computação Gráfica, Geometria Computacional, GJK, Intersecção, Polígono, Polítopo, Poliedro, Primitiva, Quatérnios, SAT, Simulação, Somas de Minkowski

Abstract

Computational Physics is utilized to create simulations in contexts such as, among others, Engeneering, Design, digital games, animation e the film industry. Despite the challenges involved in implementing physics simulations in Computer Science, there are numerous free and proprietary tools dedicated to this end that are used professionally, which continue to improve to the present day.

Techniques concerning body intersection in Computation, in particular, are continually revisited, incremented and re-imagined, with the in order to become more performant and accurate, especially in interactive simulations, calculated in real time.

This work proposes a comparison of two techniques for detection of intersection between convex rigid bodies, presenting the concepts of Analytical and Computational Geometry employed in each algorithm, and develop a library for intersection detection that utilizes one of these techniques, observing the analysis conducted and researched bibliography on each method. Once implemented, the library will be tested on a simulation for result validation and collection of data measuring the performance of its functions.

Keywords: Computational Geometry, Computer Graphics, GJK, Intersection, Minkowski Sums, Polygon, Polytope, Polyhedron, Primitive, Quaternions, SAT, Simulation

Lista de ilustrações

Figura 1 – Aplicações de Física Computacional sobre contextos distintos 13
Figura 2 – Representação geométrica de convexidade 15
Figura 3 – r_x, r_y e r_z são os "raios" da caixa retangular, a partir de seu centro . . 16
Figura 4 – r é o raio da esfera, a partir de seu centro 16
Figura 5 – r_x, r_y e r_z são os "raios" da elipsóide, a partir de seu centro 17
Figura 6 – r é o raio das semi-esferas e do cilindro da cápsula, e h é o comprimento da parte cilíndrica da forma 17
Figura 7 – r é o raio do cilindro e h é sua altura 17
Figura 8 – cada vértice da forma convexa é guardado em uma lista de vértices, cuja coordenada é relativa ao centro do corpo 17
Figura 9 – Polígono complexo de casca convexa 18
Figura 10 – Rotações eulerianas tridimensionais em ordens diferentes 20
Figura 11 – Demonstração de não-intersecção por Eixo de Separação. 24
Figura 12 – Normais das arestas de dois Polígonos 25
Figura 13 – Representação de Soma de Minkowski em $\mathbb{R}^2$ 29
Figura 14 – Vetor de direção e vértice mais distante encontrado para formar o primeiro ponto do *simplex*. 30
Figura 15 – Novo vetor de direção e vértice encontrado para formar o próximo ponto do *simplex*. 31
Figura 16 – O *simplex* é formado e pode ser verificado se contém o ponto de origem. 31
Figura 17 – Subspaços em relação a uma aresta 35
Figura 18 – Subspaços em relação a um triângulo 36
Figura 19 – Subspaços em relação a um tetraedro 38
Figura 20 – Intersecção da caixa delimitadora 49
Figura 21 – Intersecção entre os corpos 50
Figura 22 – Número máximo de iterações do laço de repetição do algoritmo por par de corpos 52
Figura 23 – Número máximo de ocorrências de diminuição do *simplex* para um **ponto** por teste entre par de corpos 53
Figura 24 – Número máximo de ocorrências de diminuição do *simplex* para uma **aresta** por teste entre par de corpos 54
Figura 25 – Número máximo de ocorrências de diminuição do *simplex* para um **triângulo** por teste entre par de corpos 54
Figura 26 – Tempo médio de execução do algoritmo **GJK** por par de corpos 55
Figura 27 – Iterações do laço de repetição e casos de redução do simplex 57
Figura 28 – Tempo médio de execução do algoritmo **GJK** por par de corpos 58

Sumário

Lista de ilustrações **9**

1 INTRODUÇÃO **13**

2 CORPOS RÍGIDOS CONVEXOS **15**
2.1 Corpos Rígidos **15**
2.2 Transformações Tridimensionais **18**
2.2.1 Transformações Geométricas 19
2.2.2 Quatérnios 19

3 ALGORITMOS DE INTERSECÇÃO ENTRE CONVEXOS **23**
3.1 Análise Assintótica **23**
3.2 Algoritmo SAT **24**
3.2.1 Pseudocódigo 26
3.2.2 Análise de Complexidade 27
3.3 Algoritmo GJK **29**
3.3.1 Somas de Minkowski 29
3.3.2 Descrição do Algoritmo 30
3.3.3 Pseudocódigo 33
3.3.4 Análise de Complexidade 39
3.4 Comparação dos Algoritmos **43**

4 IMPLEMENTAÇÃO E TESTES **45**
4.1 Estruturas de Dados **45**
4.2 Funções para Fase Ampla **46**
4.3 Funções *Support* para GJK **47**
4.4 Função de Simplex mais próximo **47**
4.5 Função GJK **48**
4.6 Simulação **49**
4.6.1 Simulação Interativa 49
4.6.2 Dados Numéricos 50
4.7 Resultados **51**
4.7.1 Primitivas 52
4.7.2 Poliedros de Casca Convexa 56

5 CONCLUSÃO E TRABALHOS FUTUROS **61**
5.1 Conclusão sobre Análises e Resultados de Testes **61**

5.2 Trabalhos Futuros 63

REFERÊNCIAS 65

A APÊNDICE 69
A.1 Operações Matemáticas 69
A.2 Enumeradores 70
A.3 Estruturas de Dados 70
A.4 Operadores e Funções Vetoriais 72
A.5 Funções *Support* para GJK 73
A.6 Funções de Colisor e Simulação 77
A.7 Função *NearestSimplex* 86
A.8 Função GJK 92

1 Introdução

A Física Computacional engloba aspectos da Geometria Analítica e da Mecânica, representados através de métodos numéricos no âmbito da Computação, para uso em simulações que imitem o comportamento dos corpos no espaço, com colisões, forças, gravidade e movimento. Alguns cenários de aplicação destas simulações são apresentadas na Figura 1.

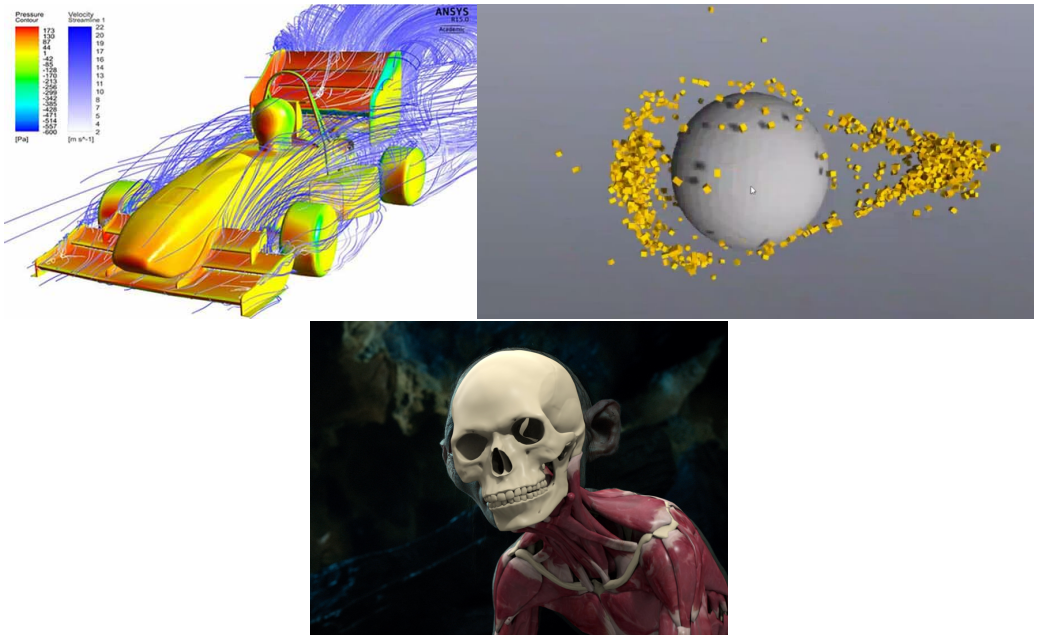


Figura 1 – Aplicações de Física Computacional sobre contextos distintos
Fonte: (Ansys Inc. (2020); Havok / Microsoft (2020); Weta Digital (2020))

Assim, as limitações da natureza discreta de um computador, de sua velocidade e capacidade de processamento (BROCHU; EDWARDS; BRIDSON, 2012) configuram um cenário que traz dificuldades em termos de rapidez de resposta da simulação e precisão dos resultados. Em particular, simulações em tempo real apresentam desafio em relação ao tempo de processamento dos dados.

Apesar disto, existem ferramentas de código aberto e gratuitas para simulação de física, utilizadas para jogos e outros cenários interativos. Pode-se citar, entre elas, a Bullet Physics, uma engine criada em C++ contendo primitivas, funções de colisão e otimizações de performance utilizando recursos de GPU e paralelização (COUMANS, 2013). No entanto, até os dias atuais, técnicas de intersecção entre corpos são revisitadas por acadêmicos, sofrem alterações, são testadas em novos contextos, e estratégias tradicionais utilizadas em algoritmos de geometria computacional são questionadas ou repensadas (MACKLIN et al., 2020).

Neste trabalho, é proposta uma implementação genérica de cálculo de intersecção entre formas geométricas convexas, com o intuito de estruturar o código de maneira unificada e organizada, e homogeneizar a performance de cada caso em relação aos demais.

Para isto, tem-se como objetivo apresentar os conceitos e princípios de Geometria aplicados nos métodos predominantemente (COUMANS, 2013; GREGORIUS, 2015) utilizados em bibliotecas, *frameworks* e *engines* de física para simulações interativas. Em seguida, estabelecer uma comparação entre estes métodos, considerando resultados apresentados pela literatura e análise de algoritmo. Por fim, o objetivo é desenvolver uma ferramenta contendo um conjunto de algoritmos, funções e estruturas de dados para detectar intersecções entre corpos rígidos convexos tridimensionais, extrair resultados de testes aleatórios de simulação e expor um sistema de visualização desta ferramenta em funcionamento. Especificamente, pretende-se:

- Comparar a eficiência e flexibilidade entre os algoritmos **SAT** e **GJK**;
- Implementar e validar as estruturas de dados e funções para simular intersecções, utilizando o algoritmo **GJK**.

O Capítulo 2 introduz os tipos de geometria e conceito de corpos rígidos trabalhados na implementação proposta, e mostra a representação das transformações destes corpos no espaço. O Capítulo 3 apresenta dois algoritmos de intersecção entre formas convexas, apresentando as análises assintóticas de cada um e a comparação entre os algoritmos. O Capítulo 4 descreve as estruturas e funções implementadas para desenvolvimento da biblioteca de intersecção, execução dos testes e simulação gráfica, e resultados obtidos de cada teste. Já no Capítulo 5 mostra as considerações finais sobre as análises e testes feitos nos capítulos anteriores, e sugestões para trabalhos futuros.

2 Corpos Rígidos Convexos

2.1 Corpos Rígidos

No contexto de intersecção entre corpos durante uma simulação interativa em Mecânica Clássica, pode-se categorizar os corpos em conjuntos tais como rígidos ou flexíveis. Para o escopo apresentado neste trabalho, faz-se um recorte focado exclusivamente sobre corpos rígidos convexos.

De acordo com BEER (et al.,2013), corpos rígidos são compreendidos como formas que não se deformam, ou seja, não esticam, contraem, expandem, torcem, etc, independente das forças internas ou externas aplicadas sobre elas. Entenda-se, portanto, que um corpo qualquer seja composto por um conjunto infinito de pontos, e sendo ele rígido, a distância entre estes pontos não se altera em qualquer contexto de uma simulação.

Desta forma, entende-se por corpo rígido uma forma que possa sofrer translação, rotação e escala, sem perder a configuração de seus limites. É importante observar que no contexto de Dinâmica de Corpos Rígidos tradicional, corpos rígidos também não variam de escala, pois são compostos de materiais que não podem se contrair ou expandir.

Para os processos de intersecção, serão estudados em particular corpos rígidos convexos. Um corpo é dito convexo se, e somente se, dados dois pontos a e b contidos dentro de seus limites estão interligados por uma aresta $\overline{ab}$ que esteja também inteiramente contida dentro destes limites (BERG et al., 1997; CHEN, 1996), como ilustrado na Figura 2.

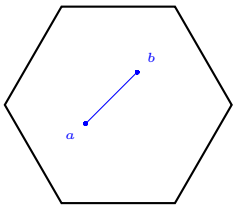


Figura 2 – Representação geométrica de convexidade
Fonte: Do próprio autor

Dentro do conjunto de corpos convexos encontram-se, então, formas primitivas, tais como caixas retangulares, elipsóides, cilindros, cápsulas, e polítopos de casca convexa, isto é, formas geométricas representadas através de números finitos de ângulos, retas e planos.

Antes de dar continuidade ao trabalho, é importante distinguir primitivas de polítopos convexos, por dois motivos:

- Primitivas podem ser formadas tanto por superfícies curvas quanto retas, enquanto polítopos são compostos apenas por arestas/faces;

- Primitivas são formas geométricas descritas por equações e parametrizações. Polítopos, por outro lado, são representados através de conjuntos de pontos, arestas e faces finitos. Polítopos não possuem fórmulas matemáticas que os descrevem.

O foco do desenvolvimento deste trabalho está nas seguintes formas: caixas retangulares, esferas, elipsóides, cápsulas, cilindros e poliedros convexos. A representação geométrica destas formas segue conforme figuras 3 a 8[1].

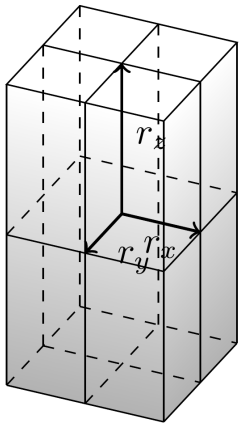


Figura 3 – r_x, r_y e r_z são os "raios" da caixa retangular, a partir de seu centro

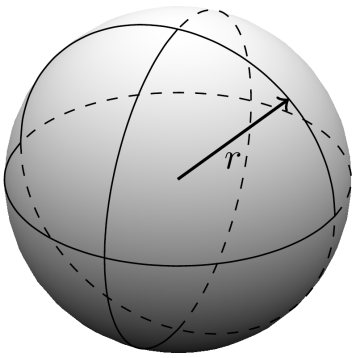


Figura 4 – r é o raio da esfera, a partir de seu centro

[1] As figuras 3 a 8 são originais deste trabalho.

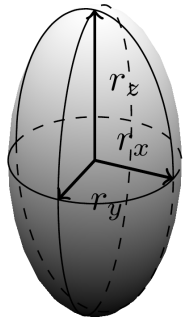


Figura 5 – r_x, r_y e r_z são os "raios" da elipsóide, a partir de seu centro

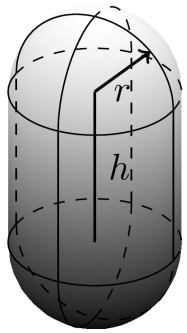


Figura 6 – r é o raio das semi-esferas e do cilindro da cápsula, e h é o comprimento da parte cilíndrica da forma

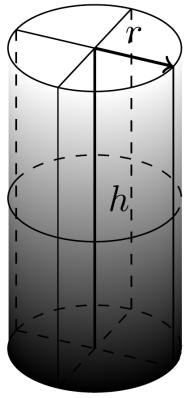


Figura 7 – r é o raio do cilindro e h é sua altura

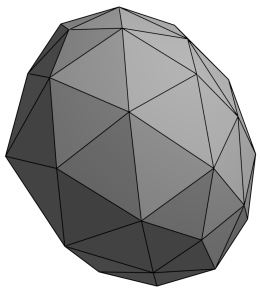


Figura 8 – cada vértice da forma convexa é guardado em uma lista de vértices, cuja coordenada é relativa ao centro do corpo

Os algoritmos de detecção de intersecção **SAT** e **GJK**, introduzidos no Capítulo 3, apresentam resultados corretos exclusivamente quando executados sob corpos convexos, conforme será demonstrado. Além disso, em simulações de colisão entre corpos côncavos, é possível particionar as formas em subformas convexas, e executar os testes vistos no Capítulo 4 para cada casca convexa resultante destes particionamentos (O'ROURKE, 1997).

Outra característica do **SAT** e do **GJK** é a corretude que eles mantém em cenários cujos corpos consistam de polítopos complexos de casca convexa, que são formas geométricas finitas em que pontos, arestas ou faces pertencentes a estas formas não compõem os limites dos corpos. Isto significa que estes elementos interceptam a geometria. No entanto, polítopos complexos podem ser compostos por uma casca (ou silhueta) convexa, vide Figura 9.

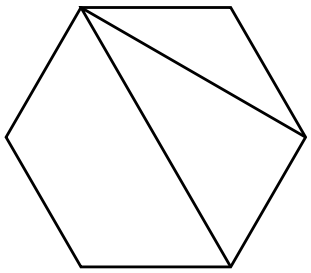


Figura 9 – Polígono complexo de casca convexa
Fonte: Do próprio autor

Esta característica de corretude, presente nos dois métodos, é fundamental para lidar com as formas geométricas descritas computacionalmente durante as simulações. Em função do que será visto quanto a Somas/Diferenças de Minkowski e/ou representações computacionais dos corpos simulados, algoritmos de detecção de colisão podem receber como entrada polítopos complexos, descritos como geometria degenerada.

2.2 Transformações Tridimensionais

Nesta seção, são descritas matematicamente as transformações dos corpos no espaço e suas limitações, para serem utilizadas nos algoritmos de detecção de intersecção estudados.

Estas fórmulas permitem representar a forma de um objeto separadamente do movimento que ele descreve no espaço, que traz duas vantagens para a máquina. Primeiramente, as transforções cumulativas aplicadas sobre o objeto desta maneira evita propagação de erros de imprecisão numérica. Em segundo lugar, tratando-se de formas descritas por conjuntos de vértices, arestas e faces, tais transformações precisam ser aplicadas a cada elemento destes conjuntos, e podem ser executadas em paralelo, utilizando técnicas de Computação Gráfica.

2.2.1 Transformações Geométricas

As transformações de corpos no espaço podem ser representadas por operações entre vetores. As transformações utilizadas nos algoritmos de intersecção entre corpos rígidos são descritas da seguinte maneira:

- **Translação:** Seja $a = (a_x, a_y, a_z)$ um ponto no espaço e $\vec{t}$ um vetor de translação. Então a translação de a pelo vetor $\vec{t}$ se dá por:

$$T(a) = (a_x + \vec{t}_x, a_y + \vec{t}_y, a_z + \vec{t}_z)$$

- **Escala:** Seja $a = (a_x, a_y, a_z)$ um ponto no espaço e $\vec{s}$ um vetor de escala. A escala de a por $\vec{s}$ é representada como:

$$S(a) = (a_x\vec{s}_x, a_y\vec{s}_y, a_z\vec{s}_z)$$

- **Rotação Euleriana:** Seja $a = (a_x, a_y, a_z)$ um ponto no espaço e θ um ângulo de rotação. As rotações de a por θ nos eixos canônicos X, Y e Z são descritas, respectivamente, como:

$$\begin{aligned} R_x(a) &= (a_x, a_y \cos\theta - a_z \sin\theta, a_y \sin\theta + a_z \cos\theta) \\ R_y(a) &= (a_x \cos\theta + a_z \sin\theta, a_y, a_z \cos\theta - a_x \sin\theta) \\ R_z(a) &= (a_x \cos\theta - a_y \sin\theta, a_x \sin\theta + a_y \cos\theta, a_z) \end{aligned}$$

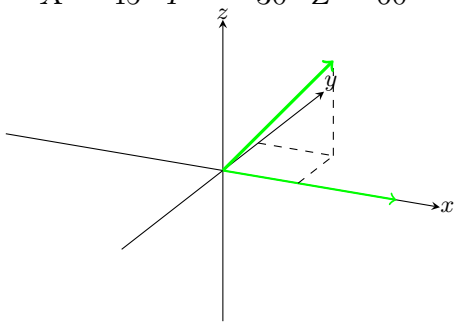
2.2.2 Quatérnios

Rotações Eulerianas apresentam, inicialmente, duas limitações para a representação de transformações de objetos no espaço tridimensional: rotações separadas por eixo não-comutativas e bloqueio de Gimbal[2].

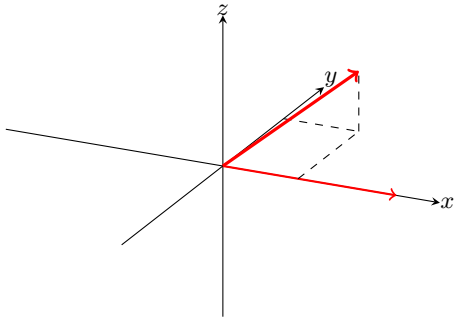
Uma rotação Euleriana envolve uma transformação de orientação cumulativa sobre as rotações anteriores, e cada rotação é descrita apenas um eixo por vez, então observa-se que uma rotação em X, Y e Z subsequentemente produz uma transformação diferente de uma rotação em X, Z e Y, por exemplo. Isto ocorre porque as rotações descrevem transformações em relação aos eixos canônicos do espaço vetorial representado. Seria necessário escolher um vetor orientado arbitrariamente no espaço e aplicar uma rotação em torno deste vetor. A Figura 10 ilustra este fenômeno.

[2]Bloqueio de Gimbal, ou *Gimbal Lock*, em inglês, é o fenômeno de sobreposição entre dois eixos de rotação.

$X = 45°\ Y = -30°\ Z = 60°$



(a) Rotação na ordem Z, depois Y e por último X do vetor $(2, 0, 0)$



(b) Rotação na ordem X, depois Y e em seguida Z do vetor $(2, 0, 0)$

Figura 10 – Rotações eulerianas tridimensionais em ordens diferentes
Fonte: Do próprio autor

O fenômeno de bloqueio de Gimbal, por outro lado, ocorre em espaços com dimensões maior ou igual a três. Tomando um exemplo em que se faça uma rotação na ordem XYZ, se o eixo Y for orientado em $\frac{\pi}{2}$, o eixo X, depois de rotacionado por Y, fica sobreposto sobre o eixo Z. Assim, quaisquer rotações sobre X e sobre Z transformam o objeto no mesmo plano, e a possibilidade de rotação no plano do eixo X se perde até que a rotação em Y seja modificada. Desta forma, usa-se como solução para rotações no espaço tridimensional(HUGHES et al., 2014) o conceito de **quatérnios**.

HAMILTON (1992) expõe um estudo envolvendo números complexos a partir de uma interpretação geométrica, estabelecendo a definição de quatérnios como números complexos compostos de 4 componentes que residem em 4 eixos geométricos distintos. Em termos matemáticos: Um quatérnio q é um número complexo dado por

$$q = (a, b, c, d) = a + bi + cj + dk$$

onde $a, b, c, d \in \mathbb{R}$ e i, j, k são componentes imaginárias. É conhecido que a visualização geométrica deste vetor é incompreensível ao olhar humano sem que haja uma interpretação ou convenção da representação destes 4 eixos e a transformação que descreve sua orientação e magnitude[3].

[3]Existem proposições de visualização variadas para quatérnios, como por exemplo em (BAEK et al., 2017).

$$i^2 = j^2 = k^2 = ijk = -1$$

$$ij = k, jk = i, ki = j$$

$$ji = -k, kj = -i, ik = -j$$

Segundo HAMILTON (1992), a soma entre números complexos é trivial. Já a multiplicação entre dois quatérnios q_1 e q_2, é

$$q_1q_2 = (a_1 + b_1i + c_1j + d_1k)(a_2 + b_2i + c_2j + d_2k) = a_1a_2 - (b_1b_2 + c_1c_2 + d_1d_2) +$$
$$a_1(b_2i + c_2j + d_2k) + a_2(b_1i + c_1j + d_1k) + i(c_1d_2 - d_1c_2) +$$
$$j(d_1b_2 - b_1d_2) + k(b_1c_2 - c_1b_2)$$

Nota-se que este produto é anticomutativo, devido às relações multiplicativas entre i, j e k. Agora, usando **Produto Escalar** e **Produto Vetorial**[4], obtemos:

$$q_1q_2 = (s_1, \vec{v_1})(s_2, \vec{v_2}) = (s_1s_2 - \vec{v_1} \cdot \vec{v_2}, s_1\vec{v_2} + s_2\vec{v_1} + \vec{v_1} \times \vec{v_2})$$

s_1 e s_2 são as partes reais e $\vec{v_1}$ e $\vec{v_2}$ são os vetores representando os 3 eixos imaginários i, j e k dos quatérnios q_1 e q_2.

Para associar quatérnio e vetor de transformação por rotação em $\mathbb{R}^3$, representa-se o ponto como um quatérnio que sai da origem do espaço geométrico até suas coordenadas, e utiliza-se suas componentes (x, y, z) como os escalares reais b, c e d que multiplicam os fatores imaginários i, j e k. Tem-se então uma coordenada $A = (x_A, y_A, z_A) \Rightarrow q_A = 0 + x_Ai + y_Aj + z_Ak = (0, \vec{v_A})$, sendo a componente real nula, para que se possa parametrizar os eixos reais originais aos quais A pertence como os eixos imaginários aos quais q_A pertence. A multiplicação entre um quatérnio q por q_A é obtida com a fórmula

$$qq_A = (s, \vec{v})(0, \vec{v_A}) = (-\vec{v} \cdot \vec{v_A}, s\vec{v_A} + \vec{v} \times \vec{v_A})$$

Rotações quaternianas, no entanto, são quadridimensionais. Segundo BAEK (et al.,2017), a tentativa de visualizar a rotação de um ponto tridimensional(A) por um quatérnio(q) apresenta variações de translação e magnitude do vetor saindo da origem em direção ao ponto(chamado acima de $\vec{A}$). É necessário então multiplicar qq_A por um quatérnio inverso a q, q^{-1}, de tal forma que $qq^{-1} = q^{-1}q = 1$, para que o quatérnio q_A, e consequentemente o vetor $\vec{A}$, seja apenas rotacionado, e não translatado e nem escalado. Particularmente, usa-se quatérnios unitários, que contém a propriedade $s^2 + \|\vec{v}\|^2 = 1$.

Para quatérnios unitários, o quatérnio inverso é igual ao conjugado, representado por $\overline{q}$, um quatérnio de mesma magnitude e direção de q, mas de sentido oposto, ou seja,

[4]Descritos na Seção A.1.

seja $q = (s, \vec{v})$, então $\overline{q} = (s, -\vec{v})$, o que torna trivial a obtenção do quatérnio q^{-1}. Observa-se que a **anticomutatividade** da multiplicação entre quatérnios pode ser representada como $q_1 q_2 = q_2 q_1^{-1}$.

Finalmente, pode-se rotacionar um ponto A em relação à origem do espaço vetorial pela fórmula $R_q(q_A) = q q_A q^{-1} = q q_A \overline{q}$. Para se rotacionar o ponto A por um ângulo θ em torno de um eixo $\vec{v}$, então, é utilizado um quatérnio $q = (\cos\frac{\theta}{2}, \sin\frac{\theta}{2}\vec{N})$, $\|\vec{N}\| = \frac{\vec{v}}{\|\vec{v}\|} = 1$, em que $\vec{N}$ tem mesma direção e sentido de $\vec{v}$. Usa-se $\frac{\theta}{2}$ pois ocorrem, na realidade, 2 rotações quaternianas, uma com q e uma com $\overline{q}$, portanto orientando A em $\frac{\theta}{2} + \frac{\theta}{2} = \theta$.

Exemplificando, seja um vetor tridimensional $\vec{a} = (1, 0, 0)$, e deseja-se rotacioná-lo em relação ao eixo $\vec{N} = (0, 0, 1)$ em $\frac{\pi}{3}$ radianos. Primeiro, o vetor $\vec{a}$ precisa ser representado em forma de quatérnio, chamado aqui de $q_a = (0, (1, 0, 0))$. Em seguida, é definido o quatérnio de rotação $q_r = (\cos\frac{\pi}{6}, \sin\frac{\pi}{6}\vec{N})$ e seu inverso $q_r^{-1} = (\cos\frac{\pi}{6}, -\sin\frac{\pi}{6}\vec{N})$. A rotação R_q do quatérnio q_a é então obtida pela operação

$$R_q(q_a) = q_r q_a q_r^{-1} = (q_r q_a) q_r^{-1}$$

Primeiro, faz-se a meia rotação da multiplicação de q_r pelo quatérnio q_a. Observa-se que a magnitude do vetor resultante desta operação (a parte imaginária do quatérnio) possui magnitude diferente do vetor $\vec{a}$ original.

$$q_r q_a = (s_{q_r} s_{q_a} - \vec{v_{q_r}} \cdot \vec{v_{q_a}}, s_{q_r} \vec{v_{q_a}} + s_{q_a} \vec{v_{q_r}} + \vec{v_{q_r}} \times \vec{v_{q_a}}) =$$
$$q_r q_a = \left(\cos\tfrac{\pi}{6}, \sin\tfrac{\pi}{6}(0,0,1)\right)\ (0, (1,0,0)) =$$
$$\left(-\sin\tfrac{\pi}{6}(0,0,1) \cdot (1,0,0), \cos\tfrac{\pi}{6}(1,0,0) + \sin\tfrac{\pi}{6}(0,0,1) \times (1,0,0)\right) =$$
$$\left(\left(0,0,-\tfrac{1}{2}\right) \cdot (1,0,0), \left(\tfrac{\sqrt{3}}{2}, 0, 0\right) + \left(0, \tfrac{1}{2}, 0\right)\right) =$$
$$\left(0, \left(\tfrac{\sqrt{3}}{2}, \tfrac{1}{2}, 0\right)\right)$$

Em seguida, é feita a segunda meia rotação multiplicando o quatérnio resultante da operação $q_r q_a$ pelo inverso de q_r, q_r^{-1}. Quando esta operação é concluída, a parte imaginária do quatérnio resultante possui mesma magnitude do vetor original $\vec{a}$.

$$(q_r q_a) q_r^{-1} = (s_{q_r q_a} s_{q_r^{-1}} - \vec{v_{q_r q_a}} \cdot \vec{v_{q_r^{-1}}}, s_{q_r q_a} \vec{v_{q_r^{-1}}} + s_{q_r^{-1}} \vec{v_{q_r q_a}} + \vec{v_{q_r q_a}} \times \vec{v_{q_r^{-1}}})$$
$$(q_r q_a) q_r^{-1} = \left(0, \left(\tfrac{\sqrt{3}}{2}, \tfrac{1}{2}, 0\right)\right) \quad \left(\tfrac{\sqrt{3}}{2}, \left(0, 0, -\tfrac{1}{2}\right)\right) =$$
$$\left(-\left(\tfrac{\sqrt{3}}{2}, \tfrac{1}{2}, 0\right) \cdot \left(0, 0, -\tfrac{1}{2}\right), \left(\tfrac{3}{4}, \tfrac{\sqrt{3}}{4}, 0\right) + \left(-\tfrac{1}{4}, \tfrac{\sqrt{3}}{4}, 0\right)\right) =$$
$$\left(0, \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}, 0\right)\right)$$

O vetor obtido pela rotação $R_q(q_a)$ é a parte imaginária do quatérnio resultante

$$a' = \left(\tfrac{1}{2}, \tfrac{\sqrt{3}}{2}, 0\right)$$

3 Algoritmos de Intersecção Entre Convexos

Este capítulo apresenta os algoritmos *Separating Axis Theorem* (ou **SAT**) e *Gilbert-Johnson-Kheerthi* (ou **GJK**). A escolha destes algoritmos para detectar interseções entre corpos rígidos dá-se baseado em bibliotecas, frameworks e engines disponíveis no mercado, tais como Bullet Physics, Box2D, PhysX e Havok. Além disso, é apresentado o pseudocódigo que descreve cada um deles, vantagens e análise assintótica de complexidade de tempo.

Primeiro, é explicada a teoria referente a análise assintótica de algoritmos, para utilizar este conceito na análise de complexidade de tempo dos algoritmos **GJK** e **SAT**.

3.1 Análise Assintótica

Algoritmos implementados em computação possuem medidas de eficiência baseadas em tempo de processamento e quantidade de memória necessária para concluírem uma tarefa com uma quantidade n de dados.

No entanto, devido ao aumento de capacidade de memória substancial das máquinas ao longo das décadas, tanto memória RAM quanto espaço de armazenamento secundário(*hard disk drive*)[1], os cálculos assintóticos de complexidade de espaço perderam parcialmente relevância diante dos de cálculos complexidade de tempo(CORMEN et al., 2009)[2].

A complexidade de tempo é medida em número de passos ou ações executadas pelo algoritmo em função de uma quantidade n de dados de entrada, e abstraída de valores constantes ou números de ciclos necessários para cada tarefa. Além disso, toma-se o maior polinômio de uma dada complexidade para a notação final, ou seja, $c_1n^2 + c_2n + c_3 = O(n^2), c_1, c_2, c_3 \in \mathbb{R}_+$.

Os cálculos de complexidade assintótica estão, comumente, dentro de uma das seguintes categorias: $1,\ \lg n,\ n,\ n \lg n,\ n^2,\ n^2 \lg n, ...,\ n^{m-1} \lg n,\ n^m,\ 2^n$ e $n!$, com $n, m \in \mathbb{N}$, e m constante. Faz-se um "abuso"(CORMEN et al., 2009; KNUTH, 1997) do uso da notação O[3], mas as notações Ω, θ e O são descritas, mais precisamente, como (CORMEN et al., 2009):

- **Notação Ω:** Para uma função qualquer $g(n)$, define-se $\Omega(g(n))$ como o conjunto

[1] Os primeiros HDDs da IBM, na década de 50, armazenavam 3.75 megabytes, e os tubos de Williams, do final da década de 40, armazenavam de 1024 a 2560 bits de dados. Em 2017, HDDs chegam a 12 terabytes e pentes de memória RAM a 64 Gigabytes.

[2] Tratando-se de complexidade de espaço polinomial. (SIPSER, 2013) possui um capítulo dedicado a problemas de complexidade de espaço NP, que está além do escopo deste projeto.

[3] Particularmente por tratar-se do limite superior da análise de pior caso.

de funções $f(n)$ para as quais $\exists\ c, n_0$ constantes positivas tais que $0 \leqslant cg(n) \leqslant f(n), \forall\ n \geqslant n_0$. $\Omega(g(n))$ é chamado de **limite inferior assintótico** de $g(n)$;

- **Notação Θ:** Para uma função qualquer $g(n)$, define-se $\Theta(g(n))$ como o conjunto de funções $f(n)$ para as quais $\exists\ c_1, c_2$ e n_0 constantes positivas tais que $0 \leqslant c_1 g(n) \leqslant f(n) \leqslant c_2 g(n), \forall\ n \geqslant n_0$. $\Theta(g(n))$ denota-se como o **limite restrito assintótico**;

- **Notação O:** Para uma função qualquer $g(n)$, define-se $O(g(n))$ como o conjunto de funções $f(n)$ para as quais $\exists\ c, n_0$ constantes positivas tais que $0 \leqslant f(n) \leqslant cg(n), \forall\ n \geqslant n_0$. Chama-se $O(g(n))$ de **limite superior assintótico** de $g(n)$.

3.2 Algoritmo SAT

A ideia que motiva a aplicação do algoritmo **SAT** é de que se dois objetos são convexos e existe uma separação entre eles, existe uma quantidade infinita de retas (em $\mathbb{R}^2$) ou planos (em $\mathbb{R}^3$) entre eles que separa o espaço geométrico em dois subespaços, cada um contendo um dos objetos, conforme o exemplo da Figura 11.

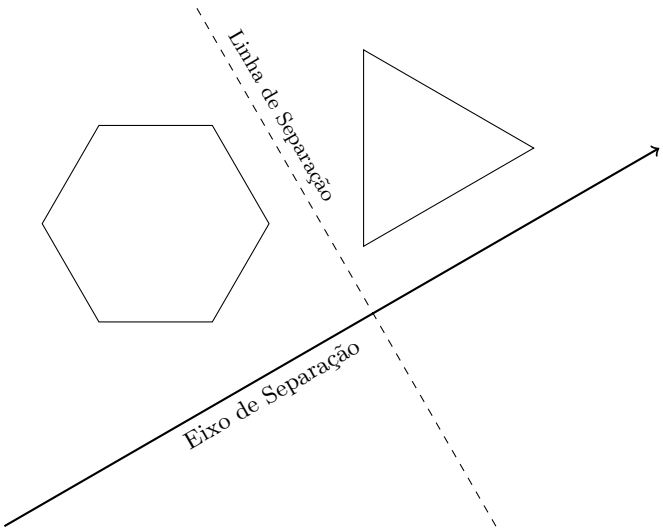


Figura 11 – Demonstração de não-intersecção por Eixo de Separação.
Fonte: Do próprio autor

Para verificar se existe pelo menos uma reta que separe os dois corpos, é preciso definir um conjunto de eixos sobre os quais serão projetados os limites mínimos e máximos de cada objeto, e verificar se existem sobreposições entre estes limites. Estes eixos serão denominados **Eixos de Separação**. No caso de dois polígonos convexos, as normais das arestas são os eixos considerados, como mostra a Figura 12.

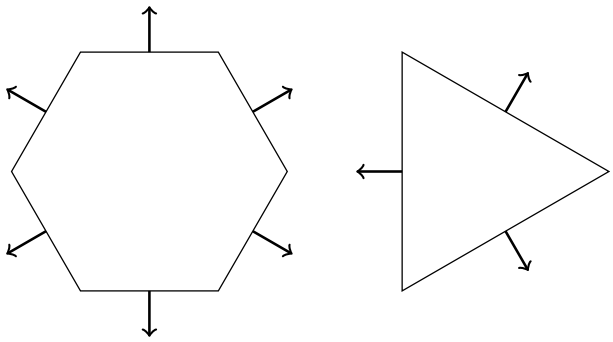


Figura 12 – Normais das arestas de dois Polígonos
Fonte: Do próprio autor

Se existir pelo menos um eixo sem sobreposição entre os pontos mínimos e máximos dos corpos projetados sobre ele, então existe uma reta separando os dois corpos, e portanto eles não se interseccionam, conforme mostra a sequência de passos que descreve o algoritmo **SAT**, a seguir:

1. Sejam dois corpos convexos A e B;
2. Considere cada vetor normal de aresta dos corpos A e B como um eixo separação;
3. Uma vez listados todos os eixos de separação, para cada eixo na lista:
   a) Encontre os vértices máximos e mínimos de cada corpo, isto é, os vértices com menor e maior produto escalar de cada corpo em relação ao eixo de separação;
   b) Obtém-se 4 valores, A-mínimo, A-máximo, B-mínimo e B-máximo, e é feita a verificação:
      i. Se A-mínimo for menor do que B-máximo **E** A-mínimo for maior do que B-mínimo, **OU** se B-mínimo for menor do que A-máximo **E** B-mínimo for maior do que A-mínimo, então continue o teste (existe uma sobreposição entre A e B neste eixo);
      ii. Caso contrário, existe uma separação entre os corpos A e B, a rotina para e retorna como **falsa** a intersecção.
4. Uma vez que exista sobreposição entre A e B em **todos** os eixos de separação, a rotina retorna como **verdadeira** a intersecção.

Conforme a descrição do algoritmo **SAT**, uma possível dificuldade é trabalhar com formas paramétricas contínuas, uma vez que a superfície de uma forma contínua possui infinitas normais. Esferas, excepcionalmente, podem ser verificadas utilizando **SAT** de maneira trivial devido às dimensões inalteradas de sua forma quando projetadas sob qualquer eixo (GREGORIUS, 2013). Elipses e elipsóides, por outro lado, são particularmente

complicadas de se testar utilizando esta rotina. Wang et al (2004) demonstram um método numérico iterativo que se baseia no Teorema do Eixo de Separação através da busca de raízes reais de igualdade entre as equações das elipsóides testadas, em $\mathbb{R}^3$.

O algoritmo possui uma outra limitação que envolve a representação computacional das formas geométricas. Se os corpos testados são polígonos ou poliedros convexos representados por conjuntos de vértices, é necessário encontrar todas as arestas e faces de cada corpo que pertençam apenas à casca convexa da geometria, se não, para N vértices pertencentes a uma forma geométrica, N^2 arestas terão suas normais listadas como eixos de separação para teste na rotina, e a fórmula indicará uma complexidade de $O(N^3)$ em $\mathbb{R}^3$ (GREGORIUS, 2013). Uma estrutura de dados que represente explicitamente as arestas e suas adjacências em um corpo é necessária para que o número de verificações do laço de repetição do algoritmo não aumente com eixos de separação desnecessários.

Conforme a descrição do **SAT**, uma vez que seja encontrada uma separação em um único eixo entre os corpos, é possível determinar que as duas formas não estejam sobrepostas, mas para concluir que existe sobreposição, é necessário que se teste exaustivamente **todos** os eixos de separação e que seja verificado que não há separação entre as formas projetadas em cada eixo, tornando-o um algoritmo de força bruta (GREGORIUS, 2013). O melhor caso de execução para **SAT** ocorre quando a primeira iteração encontra uma separação, executando o algoritmo em $O(1)$.

3.2.1 Pseudocódigo

Depois de explicado o funcionamento do **SAT**, ele é apresentado em pseudocódigo, para que seja efetuada a análise de sua complexidade assintótica de tempo. O Algoritmo 1 mostra o pseudocódigo da função principal do **SAT**.

Algoritmo 1: Separating Axis Theorem

```
1  Input Forma A, Forma B
2  Output Verdadeiro ou Falso para intersecção
3  Normals = {};
4  Normals ← A.normals ∪ B.normals;
5  for normal in Normals do
6      aMin, aMax ← minMax(A, normal);
7      bMin, bMax ← minMax(B, normal);
8      if (aMin > bMin and aMin < bMax) or
9      (bMin > aMin and bMin < aMax) then
10         continue;
11     else
12         return false;
13 return true;
```

Fonte: Adaptado de (GREGORIUS, 2013)

O Algoritmo 1 utiliza uma subrotina chamada *minMax*. Esta subrotina encontra os produtos escalares mínimo e máximo dos vértices do corpo pelo vetor, definindo quais são os limites mínimos e máximos de uma forma quando projetada sob a reta formada por este vetor. A rotina está descrita no Algoritmo 2.

Algoritmo 2: Função minMax

```
1  Input Forma A, Vetor v⃗
2  Output Menor e Maior produtos escalares entre os vértices de A e v⃗
3  aMin ← ∞;
4  aMax ← −∞;
5  for a in A.vertices do
6      dot ← a · v⃗;
7      if dot > aMax then
8          aMax ← dot;
9      if dot < aMin then
10         aMin ← dot;
11 return aMin, aMax;
```

Fonte: Adaptado de (GREGORIUS, 2013)

3.2.2 Análise de Complexidade

Sejam duas formas A e B, cada uma contendo i e j arestas, e k e l vértices, respectivamente. O **SAT** opera de acordo com a complexidade de tempo descrita no Algoritmo 3:

Algoritmo 3: Análise Assintótica: Separating Axis Theorem

```
1  Input Forma A, Forma B
2  Output Verdadeiro ou Falso para intersecção
3  Normals = {};                                          // (O(1))
4  Normals ← A.normals ∪ B.normals;                       // (O(i + j))
5  for normal in Normals do                               // (O(I + J))
6      aMin, aMax ← minMax(A, normal);                    // (O(k))
7      bMin, bMax ← minMax(B, normal);                    // (O(l))
8      if (aMin > bMin and aMin < bMax) or
9      (bMin > aMin and bMin < aMax) then                 // (O(1))
10         continue;                                      // (O(1))
11     else
12         return false;                                  // (O(1))
13 return true;                                           // (O(1))
```

Fonte: Adaptado de (GREGORIUS, 2013)

A função *minMax*, por sua vez, roda em tempo de complexidade linear. Seja A uma forma geométrica contendo k vértices, o Algoritmo 4 mostra a complexidade de tempo desta função:

Algoritmo 4: ANÁLISE ASSINTÓTICA: FUNÇÃO MINMAX

```
1  Input Forma A, Vetor v⃗
2  Output Menor e Maior produtos escalares entre os vértices de A e v⃗
3  aMin ← ∞;                                          // (O(1))
4  aMax ← −∞;                                         // (O(1))
5  for a in A.vertices do                             // (O(k))
6      dot ← a · v⃗;                                   // (O(1))
7      if dot > aMax then                             // (O(1))
8          aMax ← dot;                                // (O(1))
9      if dot < aMin then                             // (O(1))
10         aMin ← dot;                                // (O(1))
11 return aMin, aMax;                                 // (O(1))
```

Fonte: Adaptado de (GREGORIUS, 2013)

Observa-se que em função de um laço de repetição de $i + j$ iterações, contendo operações de ordem k e l, é calculada uma complexidade de tempo total do algoritmo na ordem de $O(N^2)$[4] em $\mathbb{R}^2$, sendo $N = i + j$ a soma das arestas dos corpos A e B. Segundo GREGORIUS (2013), a complexidade do algoritmo sobe para $O(N^3)$ em $\mathbb{R}^3$. Existem propostas de otimização de tempo, utilizando mapas de Gauss (GREGORIUS, 2013), ou estruturas de dados que acelerem a busca por pontos mínimos e máximos, e/ou contenham informações pré-calculadas, adjacências ou relações geométricas (ALEARDI et al., 2005).

Se os corpos forem representados apenas por vértices, a complexidade de tempo do cálculo das normais para as arestas de cada corpo sobe para $O(k^2)$ e $O(l^2)$ em $\mathbb{R}^2$. Isto ocorre pois as arestas de cada geometria são inicialmente desconhecidas, e as combinações de todos os pares de vértices precisam ser varridas.

Paralelogramos, ou outras formas que contenham arestas ou faces paralelas, permitem um corte no número de iterações do laço de repetição, visto que estes pares de arestas/faces paralelas entre si que formam eixos de separação iguais. Para isto, é necessário indicar estas propriedades de antemão na estrutura de dados das formas, dentro do algoritmo, ou criando rotinas separadas para estes casos[5].

[4] Levando em conta o polinômio de maior grau.

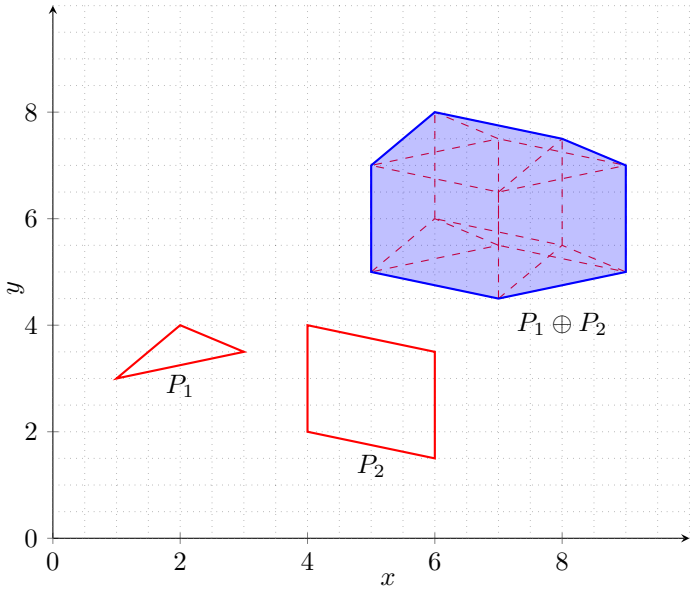
[5] Por exemplo, o caso de caixas orientadas utilizadas em árvores apresentado por (GOTTSCHALK; LIN; MANOCHA, 1996).

3.3 Algoritmo GJK

Este algoritmo é nomeado pelas iniciais dos sobrenomes de seus propositores originais, Elmer Gilbert, Daniel Johnson e Selvaraj Keerthi. A rotina parte de propriedades observadas nas operações de **Somas de Minkowski** para verificar se existem coordenadas comuns entre os corpos verificados, isto é, se alguma parte de seus volumes sofre intersecção.

3.3.1 Somas de Minkowski

Soma de Minkowski é a soma de todos os vértices de dois polítopos quaisquer P_1 e $P_2 \in \mathbb{R}^n$, com o e p vértices, respectivamente, produzindo um terceiro polítopo pertencente a $\mathbb{R}^n$, contento $o \times p$ vértices, representada como $P_1 \oplus P_2 = \sum_{i,j=1}^{o,p} a_i + b_j, \forall a_i \in P_1, b_j \in P_2$. A Figura 13 ilustra esta operação.



Os polígonos convexos P_1 e P_2 e a casca convexa resultante da soma $P_1 \oplus P_2$.

Figura 13 – Representação de Soma de Minkowski em $\mathbb{R}^2$
Fonte: Do próprio autor

A **Soma de Minkowski** pode ser utilizada para se obter uma relação entre os dois polítopos em um determinado espaço geométrico. Em particular, duas propriedades da soma são consideradas em técnicas de detecção de colisão:

I. A **Soma de Minkowski** entre duas formas convexas produz uma forma de silhueta também convexa. Matematicamente, afirma-se que a operação $\oplus$ é **fechada**[6] sobre o conjunto das formas de casca convexa;

[6]**Fechamento:** Seja # uma operação sobre um conjunto S, # é fechada em S se, e somente se, $\forall\ x, y\ \in S,\ (x \# y) \in S$.

II. A soma de um polítopo P_1 pelo reflexo de P_2(invertendo-se o sinal de todas as coordenadas de seus vértices) produz um polítopo P_3 de tal forma que se, e somente se, P_3 contém a origem do espaço geométrico, então P_1 e P_2 se interseccionam. Em termos matemáticos: Sejam $P_1, P_2 \in \mathbb{R}^n$ dois polítopos arbitrários, e seja $-P_2$ o reflexo de P_2, $P_1 \oplus (-P_2) = P_3$(ERICSON, 2005), se, e somente se, $(0, 0, ..., 0)$ estiver dentro da região delimitada por P_3, então existe pelo menos um ponto $A = (p_0, p_1, ..., p_n)$ que está dentro dos limites de P_1 e P_2, ou seja, A está dentro de $P_1 \cap P_2$. Alguns autores chamam esta operação de **Diferença de Minkowski**, representando a operação como $P_1 \ominus P_2$ (ERICSON, 2005). Além disso, P_3 pode ser denominado como **Objeto de Espaço de Configuração**(CAMERON, 1997; MONTANARI; PETRINIC; BARBIERI, 2016).

3.3.2 Descrição do Algoritmo

Utilizando as propriedades da operação de **Diferença de Minkowski**, é possível verificar se os corpos testados na operação se interseccionam. Para isso, o Algoritmo GJK opera sobre o Objeto de Espaço de Configuração (CSO - *Configuration Space Object*) para checar se a origem do espaço de simulação está contido nele. Isto é feito através de um processo iterativo de geração de um *simplex*[7] composto de vértices do **CSO** e verificando se este *simplex* contém a coordenada de origem, conforme mostra a sequência de 1 a 5, e Figura 14:

1. Escolha um vetor de direção arbitrária;
2. Procure o ponto mais distante pertencente ao **CSO** nesta direção;
3. O ponto encontrado é adicionado a uma lista de vértices que formará o *simplex* de teste;

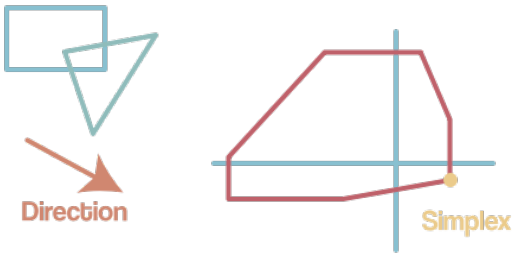


Figura 14 – Vetor de direção e vértice mais distante encontrado para formar o primeiro ponto do *simplex*.
Fonte: Kenton Hamaluik(2017)

[7]**Simplex:** É a generalização do conceito de triângulo ou tetraedro para dimensões arbitrárias. Em $\mathbb{R}^2$, um 2-*simplex* é um triângulo, e em $\mathbb{R}^3$, um 3-*simplex* é um tetraedro. Não relacionado com o método Simplex, para obtenção de solução ótima em modelos de Programação Linear.

4. O vetor passa a apontar para a direção inversa, conforme a Figura 15;

5. Repita:

   a) Procure o ponto mais distante do **CSO** nesta direção;

   b) Se o produto escalar deste ponto com o vetor direção for negativo, **pare** a rotina e retorne a intersecção como sendo **falsa**.

   c) Adicione o ponto ao *simplex*;

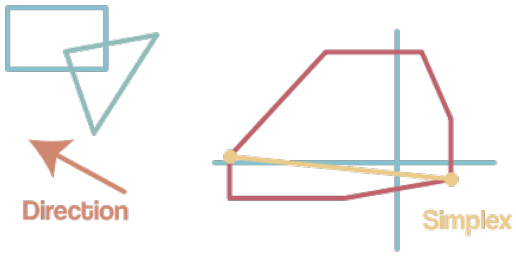


Figura 15 – Novo vetor de direção e vértice encontrado para formar o próximo ponto do *simplex*.
Fonte: Kenton Hamaluik(2017)

   d) Se o *simplex* estiver formado[8], como mostra a Figura 16, uma subrotina verifica se a origem está contida dentro dele:

      i. Se estiver, o algoritmo para e retorna como **verdadeira** a condição de intersecção;

      ii. Caso contrário, uma nova direção é escolhida para buscar um novo ponto, um vértice é removido da lista, e o laço de repetição é retomado, para formar um novo *simplex*, na tentativa de encontrar uma forma que contenha a origem.

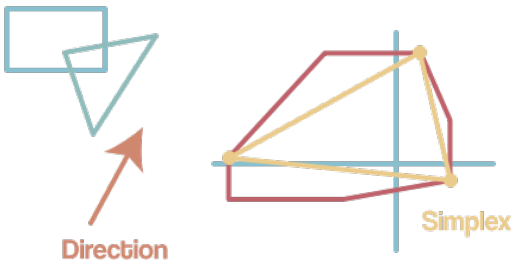


Figura 16 – O *simplex* é formado e pode ser verificado se contém o ponto de origem.
Fonte: Kenton Hamaluik(2017)

[8]Um triângulo, em $\mathbb{R}^2$.

e) Se o *simplex* ainda não estiver formado, a subrotina procura em que direção o ponto de origem está em relação à forma existente e determina como novo vetor, e o laço de repetição é retomado.

Alguns fatores do algoritmo **GJK** o tornam particularmente interessante para teste de intersecção entre formas convexas. A maior parte das operações matemáticas executadas pelo algoritmo são de produto escalar, soma e subtração, e em caso de testes entre polítopos, não existe qualquer cálculo que envolva raízes quadradas ou distância euclidiana[9].

Outro fator, para polítopos que contenham N e M vértices, respectivamente, as operações de **Soma e Diferença de Minkowski** produzem um **CSO** contendo $N \times M$ vértices, e uma busca pelo ponto mais distante do objeto em uma dada direção se torna, usando uma solução trivial, quadrática[10]. No entanto, conforme a prova demonstrada por (OLVÅNG, 2010), a função *Support* pode ser separada para cada um dos corpos testados, isto é, para dois corpos A e B, dado um vetor de direção $\vec{D}$ Support($A \ominus B$, $\vec{D}$) = Support(A, $\vec{D}$) - Support(B, $-\vec{D}$). Isto significa que a buscar pelo vértice mais distante na direção $\vec{D}$ se torna linear ($O(N + M)$).

Além do ganho de performance em separar a função *Support* para cada corpo, existe a vantagem de parametrizar esta rotina para corpos de naturezas diferentes. Formas paramétricas permitem fazer uma busca pelo ponto mais distante em uma dada direção e tempo constante, pois os limites destas formas são descritos através de fórmulas conhecidas. Não obstante, formas paramétricas curvas, tais como elipsóides, cilindros e cápsulas, podem ser representadas sem haver discretização de sua curvatura em subdivisões por retas/faces, o que significa que qualquer imprecisão de representação destes tipos de corpos se restringe a limitações de números de ponto flutuante do computador.

[9]É importante salientar que a função de Suporte descrita no pseudocódigo pode conter operações de normalização ou raízes dependendo do tipo de corpo testado.

[10]Existem algoritmos de busca por maior/menor valor em $O(NlogN)$, que não serão explorados no escopo deste trabalho.

3.3.3 Pseudocódigo

Para determinar a complexidade assintótica do **GJK**, o Algoritmo 5 mostra a descrição do pseudocódigo da função principal deste método.

Algoritmo 5: GJK Intersection

```
1  Input Forma A, Forma B, Vetor d⃗
2  Output Verdadeiro ou Falso para intersecção
3  v⃗ ← Support(A, d⃗) - Support(B, −d⃗);
4  simplex ← {v⃗};
5  d⃗ ← −v⃗;
6  while true do
7      v⃗ ← Support(A, d⃗) - Support(B, −d⃗);
8      if v⃗ · d⃗ < 0 then
9          return false;
10     simplex ← simplex ∪ v⃗;
11     simplex, d⃗, contains_origin ← NearestSimplex(simplex);
12     if contains_origin then
13         return true;
```

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

O algoritmo **GJK** utiliza duas subrotinas, a função *Support* e a função *Nearest-Simplex*. A função *Support* é utilizada para encontrar o ponto na fronteira de uma forma geométrica, em uma dada direção. A fórmula e complexidade desta função muda de acordo com o tipo de forma testada, ou seja, dependendo se o corpo é uma forma primitiva ou um polítopo de casca convexa sem fórmula de parametrização. A função *Support* para polítopos pode ser descrita conforme o Algoritmo 6:

Algoritmo 6: Função Support (polítopos)

```
1  Input Forma A, Vetor d⃗
2  Output Vértice com maior produto escalar com o vetor d⃗
3  result⃗ ← null;
4  maxVal ← −∞;
5  for v in A.vertices do
6      dot ← v · d⃗;
7      if dot > maxVal then
8          maxVal ← dot;
9          result⃗ ← v;
10 return result⃗;
```

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

Para formas paramétricas, a função *Support* pode ser implementada de acordo com a Tabela 1 a seguir:

Forma	Função Support
Segmento	$(r_x sgn(d_x), 0, 0)$
Caixa	$(r_x sgn(d_x), r_y sgn(d_y), r_z sgn(d_z))$
Disco	$r\frac{(d_x,d_y,0)}{\|(d_x,d_y,0)\|}$
Esfera	$r\vec{d}$
Elipsóide	$\frac{(r_x^2 d_x, r_y^2 d_y, r_z^2 d_z)}{\|(r_x d_x, r_y d_y, r_z d_z)\|}$
Cápsula	$Support_{segment}(\vec{d}) + Support_{sphere}(\vec{d})$
Cilindro	$Support_{segment}(\vec{d}) + Support_{disc}(\vec{d})$

Tabela 1 – Funções *Support* para formas paramétricas.
Fonte: (OLVÅNG, 2010)

A Tabela 1 mostra a parametrização da função *Support* para primitivas, onde: $\vec{d}$ é vetor de direção, d_i é a componente do vetor unitário de $\vec{d}$ na dimensão i, r_i é o "raio" da forma na dimensão i, sgn é uma função para obter o sinal de um valor (positivo ou negativo), r é o raio de uma forma quadrática (disco ou esfera).

A função *NearestSimplex* é parte central do algoritmo **GJK**. Esta rotina verifica em que subespaço particionado pelo *simplex* se encontra o ponto de origem, para verificar se a origem está contida dentro do *simplex* (um triângulo em $\mathbb{R}^2$ ou um tetraedro em $\mathbb{R}^3$), ou determinar um novo vetor de direção para fechar ou redesenhar o *simplex* que contenha este ponto de origem.

A descrição da função *NearestSimplex* está representada em 3 subrotinas nos Algoritmos 7, 8 e 9, a título de legibilidade, mas pode ser implementado em uma única função usando *switch cases* para o número de vértices que formam o *simplex*.

Algoritmo 7: FUNÇÃO NEARESTSIMPLEX (ARESTA)[a]

1 **Input** Simplex $S = \{a, b\}$
2 **Output** Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do Simplex, e novo vetor de direção $\vec{d}$
3 origin $\leftarrow (0, 0, 0)$;
4 contains_origin $\leftarrow$ false;
5 $A \leftarrow b$; `// declarar vértices em ordem inversa`
6 $B \leftarrow a$;
7 $\vec{ab} \leftarrow B - A$;
8 $\vec{a0} \leftarrow origin - A$;
9 **if** $\vec{ab} \cdot \vec{a0} > 0$ **then**
10 $\quad S \leftarrow \{A, B\}$; `// S se torna aresta AB`
11 $\quad \vec{d} \leftarrow \vec{ab} \times \vec{a0} \times \vec{ab}$;
12 **else**
13 $\quad S \leftarrow \{A\}$; `// S se torna o vértice A`
14 $\quad \vec{d} \leftarrow \vec{a0}$;
15 **return** S, contains_origin, $\vec{d}$;

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

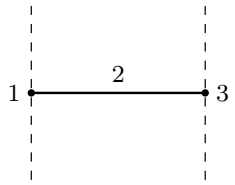


Figura 17 – Subspaços em relação a uma aresta
Fonte: Do próprio autor

O Algoritmo 7 descreve a função *NearestSimplex* para um simplex que corresponde a uma aresta, ou 1-*simplex*. A Figura 17 ilustra como o espaço é subdividido em 3 subespaços onde o ponto de origem pode residir.

Algoritmo 8: FUNÇÃO NEARESTSIMPLEX (TRIÂNGULO)[a]

```
1  Input Simplex S = {a, b, c}
2  Output Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do
     Simplex, e novo vetor de direção d⃗
3  origin ← (0, 0, 0);
4  contains_origin ← false;
5  A ← c;                                  // declarar vértices em ordem inversa
6  B ← b;
7  C ← a;
8  ab⃗ ← B − A;
9  ac⃗ ← C − A;
10 abc⃗ ← ab⃗ × ac⃗;
11 a0⃗ ← origin − A;
12 if (abc⃗ × ac⃗) · a0⃗ > 0 then
13     if ac⃗ · a0⃗ > 0 then
14         S ← {A, C};                     // S se torna a aresta AC
15         d⃗ ← ac⃗ × a0⃗ × ac⃗;
16     else
17         run NearestSimplex for S = {A, B};
18 else
19     if (ab⃗ × abc⃗) · a0⃗ > 0 then
20         run NearestSimplex for S = {A, B};
21     else
22         if abc⃗ · a0⃗ > 0 then
23             S ← {A, B, C};              // S se torna o triângulo ABC
24             d⃗ ← abc⃗;
25         else
26             S ← {A, C, B};              // S se torna o triângulo ACB
27             d⃗ ← −abc⃗;
28 return S, contains_origin, d⃗;
```

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

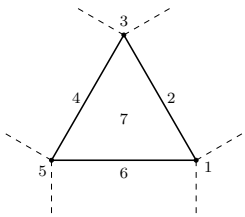


Figura 18 – Subspaços em relação a um triângulo
Fonte: Do próprio autor

O Algoritmo 8 descreve em pseudocódigo a função *NearestSimplex* para um *simplex* formado por 3 pontos que fecham um triângulo, ou 2-*simplex*. É importante observar que as linhas 17 e 20 rodam a versão do Algoritmo 7, em que um dos pontos do triângulo é descartado, e o simplex passa a ser uma aresta. A Figura 18 ilustra como o triângulo particiona o espaço em 6 subespaços onde o ponto de origem pode estar contido.

Algoritmo 9: FUNÇÃO NEARESTSIMPLEX (TETRAEDRO)[a]

1 **Input** Simplex $S = \{a, b, c, d\}$
2 **Output** Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do Simplex, e novo vetor de direção $\vec{d}$
3 origin $\leftarrow (0, 0, 0)$;
4 contains_origin $\leftarrow$ false;
5 $A \leftarrow d$; `// declarar vértices em ordem inversa`
6 $B \leftarrow c$;
7 $C \leftarrow b$;
8 $D \leftarrow a$;
9 $\vec{ab} \leftarrow B - A$;
10 $\vec{ac} \leftarrow C - A$;
11 $\vec{ad} \leftarrow D - A$;
12 $\vec{a0} \leftarrow origin - A$;
13 $\vec{abc} \leftarrow \vec{ab} \times \vec{ac}$;
14 $\vec{acd} \leftarrow \vec{ac} \times \vec{ad}$;
15 $\vec{adb} \leftarrow \vec{ad} \times \vec{ab}$;
16 **if** $\vec{abc} \times \vec{a0} < 0$ **then**
17 | **run** *NearestSimplex* for $S = \{A, B, C\}$;
18 **else if** $\vec{acd} \times \vec{a0} < 0$ **then**
19 | **run** *NearestSimplex* for $S = \{A, C, D\}$;
20 **else if** $\vec{adb} \times \vec{a0} < 0$ **then**
21 | **run** *NearestSimplex* for $S = \{A, D, B\}$;
22 **else**
23 | contains_origin $\leftarrow$ true; `// CSO contém a origem`
24 **return** S, contains_origin, $\vec{d}$;

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

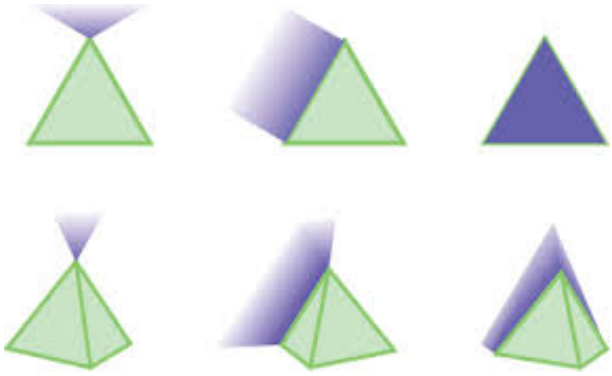


Figura 19 – Subspaços em relação a um tetraedro
Fonte: (MONTANARI; PETRINIC; BARBIERI, 2016)

O Algoritmo 9 descreve a função *NearestSimplex* para um tetraedro, ou 3-simplex. Neste caso, é possível verificar se o tetraedro formado contém ou não a origem do espaço e definir se existe uma colisão entre os dois corpos. Se o tetraedro não contiver a origem, o simplex perde um de seus pontos e torna-se um triângulo, e a rotina principal procura um novo vértice para tentar formar um tetraedro que contenha o ponto de origem. A Figura 19 mostra os 14 subespaços particionados pelo tetraedro em que o ponto de origem pode se encontrar, caso ele não esteja contido dentro do *simplex*.

3.3.4 Análise de Complexidade

Sejam dois polítopos de casca convexa A e B, cada um formado por i e j vértices, respectivamente. O Algoritmo 10 mostra a complexidade de tempo do **GJK**:

Algoritmo 10: Análise Assintótica: GJK Intersection

```
1  Input Forma A, Forma B, Vetor d⃗
2  Output Verdadeiro ou Falso para intersecção
3  v⃗ ← Support(A, d⃗) - Support(B, −d⃗);                    // O(i + j)^a
4  simplex ← {v⃗};                                          // O(1)
5  d⃗ ← −v⃗;                                                // O(1)
6  while true do                                            // O(1)^b
7      v⃗ ← Support(A, d⃗) - Support(B, −d⃗);                // O(i + j)
8      if v⃗ · d⃗ < 0 then                                   // O(1)
9          return false;                                    // O(1)
10     simplex ← simplex ∪ v⃗;                               // O(1)
11     simplex, d⃗, contains_origin ← NearestSimplex(simplex);  // O(1)
12     if contains_origin then                              // O(1)
13         return true;                                     // O(1)
```

[a]Para casos utilizando polítopos de casca convexa não parametrizáveis.
[b]Este laço, na prática, ocorre dentro de um limite de vezes e independe da quantidade de vértices dos corpos testados (CAMERON, 1997).

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

A função *Support* apresenta um comportamento de complexidade linear para polítopos convexos não paramétricos. Seja uma forma A contendo i vértices, e um vetor $\vec{d}$ de direção, a rotina opera conforme a análise descrita no Algoritmo 11:

Algoritmo 11: Análise Assintótica: Função Support (polítopos)

```
1  Input Forma A, Vetor d⃗
2  Output Vértice com maior produto escalar com o vetor d⃗
3  result⃗ ← null;                                          // O(1)
4  maxVal ← −∞;                                             // O(1)
5  for v in A.vertices do                                   // O(i)
6      dot ← v · d⃗;                                         // O(1)
7      if dot > maxVal then                                 // O(1)
8          maxVal ← dot;                                    // O(1)
9          result⃗ ← v;                                     // O(1)
10 return result⃗;                                          // O(1)
```

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

É importante observar que as implementações da função *Support* para formas paramétricas utilizam fórmulas de tempo constante para encontrar o ponto mais distante na direção $\vec{d}$, tornando a função *Support*, para estes casos, uma rotina de complexidade de tempo $O(1)$ (OLVÅNG, 2010).

As 3 implementações (para arestas, triângulos e tetraedros) da função *NearestSimplex* também ocorrem em tempo constante, como pode-se observar pelas análises nos algoritmos 12, 13 e 14:

Algoritmo 12: Análise Assintótica: Função NearestSimplex (Aresta)[a]

1 **Input** Simplex $S = \{a, b\}$
2 **Output** Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do Simplex, e novo vetor de direção $\vec{d}$
3 origin $\leftarrow (0, 0, 0)$; // $O(1)$
4 contains_origin $\leftarrow$ false; // $O(1)$
5 $A \leftarrow b$; // $O(1)$
6 $B \leftarrow a$; // $O(1)$
7 $\vec{ab} \leftarrow B - A$ // $O(1)$
8 ; $\vec{a0} \leftarrow origin - A$ // $O(1)$
9 ; **if** $\vec{ab} \cdot \vec{a0} > 0$ **then** // $O(1)$
10 | $S \leftarrow \{A, B\}$; // $O(1)$
11 | $\vec{d} \leftarrow \vec{ab} \times \vec{a0} \times \vec{ab}$; // $O(1)$
12 **else**
13 | $S \leftarrow \{A\}$; // $O(1)$
14 | $\vec{d} \leftarrow \vec{a0}$; // $O(1)$
15 **return** S, contains_origin, $\vec{d}$; // $O(1)$

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

Algoritmo 13: Análise Assintótica: Função NearestSimplex (Triângulo)[a]

```
1  Input Simplex S = {a, b, c}
2  Output Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do
     Simplex, e novo vetor de direção d⃗
3  origin ← (0, 0, 0);                                          // O(1)
4  contains_origin ← false;                                     // O(1)
5  A ← c;                                                       // O(1)
6  B ← b;                                                       // O(1)
7  C ← a;                                                       // O(1)
8  ab⃗ ← B − A;                                                  // O(1)
9  ac⃗ ← C − A;                                                  // O(1)
10 abc⃗ ← ab⃗ × ac⃗;                                               // O(1)
11 a0⃗ ← origin − A;                                             // O(1)
12 if (abc⃗ × ac⃗) · a0⃗ > 0 then                                  // O(1)
13     if ac⃗ · a0⃗ > 0 then                                      // O(1)
14         S ← {A, C};                                          // O(1)
15         d⃗ ← ac⃗ × a0⃗ × ac⃗;                                   // O(1)
16     else
17         run NearestSimplex for S = {A, B};                   // O(1)
18 else
19     if (ab⃗ × abc⃗) · a0⃗ > 0 then                              // O(1)
20         run NearestSimplex for S = {A, B};                   // O(1)
21     else
22         if abc⃗ · a0⃗ > 0 then                                 // O(1)
23             S ← {A, B, C};                                   // O(1)
24             d⃗ ← abc⃗;                                         // O(1)
25         else
26             S ← {A, C, B};                                   // O(1)
27             d⃗ ← −abc⃗;                                        // O(1)
28 return S, contains_origin, d⃗;                                // O(1)
```

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

Algoritmo 14: Análise Assintótica: Função NearestSimplex (Tetraedro)[a]

```
1  Input Simplex S = {a, b, c, d}
2  Output Simplex S atualizado, Verdadeiro ou Falso para origem contida dentro do
     Simplex, e novo vetor de direção d⃗
3  origin ← (0, 0, 0);                                   // O(1)
4  contains_origin ← false;                              // O(1)
5  A ← d;                                                // O(1)
6  B ← c;                                                // O(1)
7  C ← b;                                                // O(1)
8  D ← a;                                                // O(1)
9  ab⃗ ← B − A;                                           // O(1)
10 ac⃗ ← C − A;                                           // O(1)
11 ad⃗ ← D − A;                                           // O(1)
12 a0⃗ ← origin − A;                                      // O(1)
13 abc⃗ ← ab⃗ × ac⃗;                                        // O(1)
14 acd⃗ ← ac⃗ × ad⃗;                                        // O(1)
15 adb⃗ ← ad⃗ × ab⃗;                                        // O(1)
16 if abc⃗ × a0⃗ < 0 then                                  // O(1)
17 |  run NearestSimplex for S = {A, B, C};               // O(1)
18 else if acd⃗ × a0⃗ < 0 then                             // O(1)
19 |  run NearestSimplex for S = {A, C, D};               // O(1)
20 else if adb⃗ × a0⃗ < 0 then                             // O(1)
21 |  run NearestSimplex for S = {A, D, B};               // O(1)
22 else
23 |  contains_origin ← true;                            // O(1)
24 return S, contains_origin, d⃗;                         // O(1)
```

[a] Em $\mathbb{R}^3$.

Fonte: Adaptado de (MONTANARI; PETRINIC; BARBIERI, 2016)

As implementações da função *NearestSimplex* independem da quantidade de vértices dos corpos testados no algoritmo **GJK**, pois em todos os testes, apenas um máximo de 4 vértices (um tetraedro), é verificado se contém ou não o ponto de origem do espaço vetorial da simulação, e portanto, se o **CSO** contém a origem. Este tipo de verificação pode ocorrer devido a algumas propriedades geométricas apresentadas pela operação de **Diferença de Minkowski** e propriedades de formas com cascas convexas.

A **Diferença de Minkowski** é uma operação fechada sob o conjunto de formas de casca convexa, portanto pode-se concluir que o **CSO** durante o teste de intersecção

entre duas formas de casca convexa também é uma forma de casca convexa. Logo, infere-se que:

1. Sejam A e B dois corpos de casca convexa, $CSO = A \ominus B$ o Objeto de Configuração do Espaço, e $O = (0, 0, 0)$ o ponto de origem do espaço da simulação:
2. $i, j \in CSO \implies \overline{ij} \in CSO$;
3. $\therefore \overline{ij}, \overline{ik}, \overline{jk} \in CSO \implies \triangle ijk \in CSO$;
4. $\therefore \triangle ijk, \triangle ijl, \triangle jkl \in CSO \implies ijkl \in CSO$, seja $ijkl$ um tetraedro formado por $\triangle ijk, \triangle ijl e \triangle jkl$;
5. $\therefore O \in ijkl \implies O \in CSO$, *c.q.d.*

Devido às complexidades de tempo constantes da função *NearestSimplex* e do laço de repetição na rotina, o algoritmo **GJK** possui uma complexidade de tempo de $O(N)$, seja $N = i + j$ a soma dos vértices das polítopos convexos A e B, em que as funções *Support* operam em tempo $O(i)$ e $O(j)$, respectivamente. Para formas paramétricas, em que as funções *Support* de cada forma opera em tempo constante, o algoritmo **GJK** cai para complexidade de tempo também constante. CAMERON (1997) propõe uma estratégia com pequenas modificações ao algoritmo original que produz resultados "[...] de complexidade de tempo quase constante". Existem diversos artigos de sugestão de melhorias em performance (CAMERON, 1997; MONTANARI; PETRINIC; BARBIERI, 2016) propostas ao algoritmo **GJK**, desde sua proposição inicial em 1988.

O algoritmo não sofre um aumento polinomial de complexidade de tempo quando executado em $\mathbb{R}^3$, comparado a sua execução em $\mathbb{R}^2$, como é possível observar pela análise da rotina e suas subrotinas. Além disto, a separação da função *Support* para cada corpo testado no algoritmo permite uma implementação genérica do **GJK**, isolando para cada corpo do teste sua própria rotina para encontrar o ponto mais distante em uma dada direção.

3.4 Comparação dos Algoritmos

Ambos os algoritmos **SAT** e **GJK** são ferramentas utilizadas para detectar intersecções entre formas geométricas em ferramentas proprietárias e gratuitas (COUMANS, 2013) utilizadas no mercado de aplicativos contendo simulação interativa. Algumas soluções sugerem a implementação conjunta dos dois métodos (GREGORIUS, 2015), ou migraram de um método para o outro (COUMANS, 2013). HE et al (2019) propõe inclusive a utilização da *Bullet Physics* para executar a simulação de "[...]partículas realísticas", ao

invés de utilizar o método tradicional de elementos discretos, uma aplicação no campo de computação de física científica.

Do ponto de vista teórico, o algoritmo **SAT** é um método de compreensão trivial, pois entender que exista uma reta ou plano que separe dois objetos que não se interseccionam é visualmente intuitivo.

Com relação a complexidade de tempo, o algoritmo **GJK** é mais performático do que o **SAT**, com exceção de testes feitos com corpos de colisão compostos de quantidades baixas de vértices, tais como caixas de colisão rotacionadas(HORNUS, 2017). A margem de diferença de velocidade entre eles aumenta em $\mathbb{R}^3$, em que o **GJK** se mantém numa complexidade de tempo linear, enquanto o **SAT** sobe de uma complexidade quadrática para cúbica. Em contrapartida, embora ambos os algoritmos tenham margem para paralelização, o algoritmo **SAT** se beneficia mais com isto (COUMANS, 2013). Este fator se torna mais relevante com a criação de CPUs com maior número de *threads* e com a facilitação do uso de paralelismo em GPU, utilizando bibliotecas e *frameworks* como OpenCL e CUDA.

A maneira como os corpos são representados também torna o algoritmo **GJK** mais robusto, que funciona na mesma complexidade de tempo conjuntos de vértices, ao passo em que o método **SAT** se torna assintoticamente mais complexo se as formas não forem representadas especificamente através de arestas ou faces. São utilizadas estruturas com ponteiros de adjacências e redundâncias de variáveis (HOUSE, 2011) para tornar o algoritmo **SAT** viável em simulações interativas com malhas densas (GREGORIUS, 2013).

Além disso, a capacidade de generalização do algoritmo **GJK** para formas paramétricas e polítopos convexos é uma vantagem sobre o **SAT**. Enquanto a rotina central do **GJK** se mantém a mesma, apenas as funções *Support* de cada corpo de colisão precisa ser definida de acordo com o tipo de forma, o que pode ser facilmente organizado nas próprias classes ou estruturas dos corpos. O **SAT**, por outro lado, precisa ser tratado de forma diferenciada para lidar com casos específicos, tais como formas elipsóides orientadas (MALAPALLY, 2016).

Esta análise inicial de cada algoritmo, levando em consideração os resultados e observações de trabalhos anteriores, indica uma predileção em performance e generalização do algoritmo **GJK** como método mais eficiente para implementação de uma ferramenta de testes intersecção precisos entre corpos rígidos convexos, para ser utilizada internamente em uma biblioteca ou *framework* de simulação de física para aplicações interativas.

Embora ambos os algoritmos sejam de complexidade polinomial, eles são avaliados para aplicação em uma biblioteca de uso em simulações interativas, que são executadas em tempo real. Isto significa que o grau de complexidade de cada algoritmo interfere na sensivelmente performance da simulação. Levando isto em conta, o algoritmo **GJK** torna-se a escolha mais viável para a implementação da biblioteca proposta.

4 Implementação e Testes

Neste capítulo, é explicada a arquitetura da biblioteca de intersecção entre corpos desenvolvida. Todas as funções e estruturas foram desenvolvidas inicialmente na linguagem C e depois atualizadas para C++, e implementadas apenas com o auxílio das bibliotecas math.h e stdlib.h. Sua descrição por extenso em sintaxe de código encontra-se no Apêndice.

4.1 Estruturas de Dados

Foram criadas algumas estruturas de dados específicas para representação de vetores e quatérnios para auxiliar descrição dos corpos de colisão e suas transformações. Para evitar problemas sobre escopo de ponteiros estáticos, foram declarados explicitamente todos os componentes de cada *struct*.

Arrays em C e C++ são ponteiros estáticos e perdem referência quando são declarados dentro de um escopo específico do código. Por exemplo, quando declarados dentro de uma função, seria necessário usar alocação e liberação dinâmica de memória, que pode agregar complexidade de gerenciamento de memória desnecessários para este contexto. A estrutura de dados *v3* é utilizada pela biblioteca para representar tanto vetores tridimensionais como vértices(ou pontos) no espaço. A implementação destas estruturas estão na Seção A.3 do Apêndice.

Foram criados dois enumeradores para determinar, durante a geração dos corpos de colisão, que tipo de geometria está sendo criada, como e quais variáveis representam este corpo, e que tipo de comportamento ele terá na simulação[1]. A Seção A.2 do Apêndice mostra como estão declarados estes enumeradores.

O enumeradores *shapeType* e *behaviorType* são utilizados para definir qual função *Support* será mapeada ao corpo de colisão, e o *shapeType* define qual das estruturas mostradas na Seção A.3 do Apêndice representará as informações geométricas deste corpo. Os vértices *min* e *max* são guardados para definir os limites mínimo e máximo da caixa delimitadora de um corpo[2], e são utilizados para a "fase ampla"[3] de colisão entre os objetos.

Uma estrutura genérica de corpo de colisão foi criada contendo o vetor de translação

[1] O comportamento definido para os corpos é utilizado em pontos do código fora da parte de intersecção, mas influencia as transformações que ocorrem para cada corpo.

[2] Do inglês *Bounding Box*. Em particular, as caixas delimitadoras são **AABBs**(*Axis-Aligned Bounding Boxes*, ou Caixas Delimitadoras Alinhadas aos Eixos).

[3] *Broadphase Detection*, utilizado para fazer estimativas iniciais grosseiras de intersecção entre os corpos antes de partir para a fase de precisão.

do corpo, o quatérnio de rotação, um ID para checagem de auto-colisão, um ponteiro para a função *Support*, e outras informações pertinentes para teste e atualização da simulação, conforme a Seção A.3 do Apêndice. A título de escopo deste trabalho, os atributos de direção, velocidade/aceleração linear e angular, velocidades terminais, massa, quociente de restituição, e fricção não estão descritos na declaração da estrutura *colBody3d* no Apêndice.

Além disso, foi criada uma estrutura de dados da simulação, descrita na Seção A.3 do Apêndice, para centralizar as referências de todos os corpos que são criados no programa e facilitar a implementação de uma rotina geral de testes de colisão. A criação desta estrutura foi pensada para futuramente conter informações de direção e força de gravidade, arrasto linear e angular e controle de velocidade da simulação.

Para descrever as rotações dos corpos de colisão, funções de multiplicação entre quatérnios foram implementadas. Funções e operadores para vetores de 3 dimensões também foram criados para serem utilizados com os algoritmos de intersecção implementados.

Para as funções vetoriais e quaternianas, operações de soma e subtração entre vetores, além de produto entre vetor e escalar e multiplicação entre quatérnios, foram implementadas como sobrecargas de operadores aritméticos. A sequência de operações destas naturezas que ocorre nas funções de interesecção se torna mais legível utilizando operadores aritméticos comuns.

Em contra partida, as implementações de produto vetorial e produto escalar entre vetores foram mantidas como funções tradicionais ao invés de sobrecargas, para não confundir a interpretação da leitura destas operações no código. Também foram implementadas funções de cálculo de magnitude, normalização e de vetores para serem utilizadas nas funções *Support* específicas de certas formas paramétricas. Estas funções e operadores vetoriais estão descritos na Seção A.4 do Apêndice.

4.2 Funções para Fase Ampla

A despeito da eficácia de um algoritmo como o **GJK**, é desnecessário utilizá-lo para testar intersecção entre formas geométricas que estão distantes uma da outra. A fase ampla da simulação serve para fazer verficação grosseira para validar se os limites das caixas dos corpos alinhadas aos eixos ordenados se interseccionam, e depois fazer uma verificação aprimorada com o **GJK**, se necessário. Esta estratégia é implementada na biblioteca *Bullet Physics* (COUMANS, 2012), por exemplo.

Para executar uma fase ampla da simulação, é necessário que cada corpo tenha vértices *min* e *max* atualizados para desenhar suas Caixas de Delimitação Alinhadas aos Eixos (*Axis-Aligned Bounding Box*, ou **AABB**). Além disso, antes de entrar na fase estreita (onde ocorrem os testes refinados de colisão), é preciso que haja um teste de colisão

AABB entre as caixas dos corpos. Ambas funções de desenho e teste colisão das caixas delimitadoras estão descritos em código na Seção A.6 do Apêndice.

4.3 Funções *Support* para GJK

Conforme apresentado no Capítulo 3, polítopos e primitivas possuem definições distintas para a função *Support*, principalmente levando em conta a precisão desta rotina para formas contendo curvas que sejam representadas através de equações ou funções paramétricas. Além da preocupação em não discretizar a curvatura destas primitivas, existe um ganho de performance computacional por serem funções de complexidade de tempo de ordem constante.

Na Seção A.5 do Apêndice estão descritas, para cada tipo de corpo, duas implementações distintas. Isto foi feito para separar as transformações sofridas pelos corpos de suas funções *Support*, a título de organização.

Como a estrutura *colBody3d* possui um ponteiro de função para *Support*, é possível associar o método específico para o tipo de corpo durante sua inicialização. Assim, a implementação do algoritmo GJK não precisa detectar tipos de forma para selecionar qual rotina de *Support* utilizar, passar por *switch cases*, e nem precisa trabalhar com *casts* de tipo de corpo. Além disso, todas as transformações são calculadas apenas por operações entre vetores e quatérnios. Logo, a biblioteca não requer qualquer operação de transformação matricial.

4.4 Função de Simplex mais próximo

A função *NearestSimplex* foi implementada em uma única rotina, que trata todos os tipos de *simplex* através de um *switch case* baseado no número de vértices do *simplex* de entrada. O *simplex* é representado como um *array* de 4 elementos e o número de vértices que ele contém fica guardado em uma variável separada.

Esta função atualiza o vetor de direção, a condição de intersecção e a própria configuração e número de vértices do simplex. Assim, foi decidido passar a referência do simplex e seu número de vértices, e do vetor direção, como valores de entrada para serem atualizados durante a função. Por fim, o retorno da função é um booleano de verdadeiro ou falso, que pode ser atribuído a uma variável *contains_origin* criada na própria função **GJK**.

Esta função não verifica todos os subespaços separados pelo simplex, como visto no Capítulo 3, pois devido à maneira como os vértices do simplex são encontrados, é impossível que o ponto de origem esteja em alguns destes subespaços, como mostra a sequência de 1 a 4:

1. Se o primeiro vértice encontrado tiver um produto escalar com o vetor direção menor do que 0, pode-se concluir que a origem está mais longe do que este ponto nesta direção, e não existe nenhuma maneira do **CSO**, portanto o algoritmo pode parar e retornar a intersecção como falsa;

2. O próximo vértice encontrado, agora na direção contrária, também precisa ter produto escalar com este vetor maior do que 0, se não o **CSO** não é capaz de conter a origem. Desta forma, um teste do *NearestSimplex* para uma aresta só é capaz de devolver um vetor que esteja no subespaço entre os dois pontos que formam este segmento;

3. O triângulo formado por 3 vértices a, b e c que foram encontrados da mesma forma que nos passos anteriores só permite que o ponto de origem esteja em um de 4 subespaços: em direção a uma das normais do triângulo $\triangle abc$, para fora do triângulo em direção à aresta $\overline{ab}$, ou para forma do triângulo na direção da aresta $overlineac$;

4. Por fim, o tetraedro formado por 4 vértices a, b, c e d só pode conter a origem nos subespaços dos 3 novos triângulos gerados pela ligação do simplex anterior com o novo vértice encontrado, e não pode estar contido no subespaço fora do tetraedro na direção deste último vértice.

A implementação da função *NearestSimplex* está descrita na Seção A.7 do Apêndice.

4.5 Função GJK

A implementação feita para o algoritmo **GJK**, presente na Seção A.8 do Apêndice, apresenta uma única diferença em relação ao algoritmo original mostrado no Capítulo 3. Seja a o ponto encontrado através da função *Support* em uma direção $\vec{d}$ **qualquer** e $\vec{a} = a - (0, 0, 0)$, se $\vec{a} \cdot \vec{d} < 0$, significa que o ponto de origem está mais distante em $\vec{d}$ do que o ponto a, então é impossível que o **CSO** contenha a origem, e portanto a rotina não precisa continuar e retorna a intersecção como falsa.

É importante observar que, originalmente, o algoritmo **GJK** não é simplesmente uma rotina para definir se dois corpos se interseccionam, mas também um método de calcular a menor distância entre estes corpos, através do refinamento do *simplex* que esteja mais próximo do ponto de origem (GILBERT; JOHNSON; KEERTHI, 1988).

Este dado pode ser utilizado, em caso de intersecção, para correção de posicionamento dos corpos, a fim de simular um fenômeno de colisão física. No caso de não intersecção entre corpos, a distância tem utilidade na implementação de técnicas de Detecção Contínua de Colisão, tais como *Conservative Advancement* (ZHANG; LEE; KIM, 2006). (CATTO, 2013) apresenta a implementação de uma variação de *Conservative Advancement* utilizando **GJK**.

No entanto, diversas variações do algoritmo **GJK** foram desenvolvidas desde sua proposição (OLVÅNG, 2010), e para fins apenas de detecção de colisão, foi escolhida implementação que não calcule a menor distância entre os corpos, e sem necessidade de haver refinamento do simplex. Segundo OLVANG (2010), existem soluções separadas para obter estes dados (por exemplo, *Expanding Polytope Algorithm*) que promovem melhorias de performance para a simulação como um todo.

4.6 Simulação

4.6.1 Simulação Interativa

Para visualizar uma simulação contendo a biblioteca de intersecção, foi utilizada uma plataforma gráfica própria, ainda em desenvolvimento, feita para criar experiências interativas contendo física. A plataforma foi iniciada em OpenGL e recentemente migrada para Vulkan, e executa em cima de uma biblioteca chamada GLFW, para geração de contexto de janela multiplataforma, e reconhecimento de *inputs* de teclado e mouse.

A plataforma suporta a criação de corpos de colisão em forma de caixas retangulares, esferas, elipsóides, cápsulas, cilindros e cascas convexas. Foi criado para a simulação um corpo que pode ser deslocado e rotacionado no espaço, para visualizar a detecção de intersecção com outros objetos através de cores. A caixa delimitadora do corpo interativo é desenhada em forma de uma caixa retangular aramada roxa.

A detecção de intersecção entre os corpos ocorre em duas fases:

- **Fase Ampla:** Quando as caixas delimitadoras dos corpos colidem, o corpo controlável se torna amarelo, como mostra a Figura 20 para indicar que o teste de colisão **AABB** retornou verdadeiro e o teste em **GJK** se inicia;

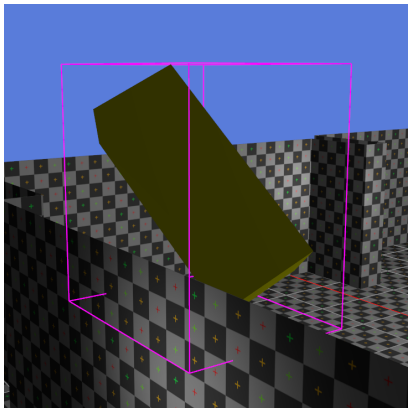


Figura 20 – Intersecção da caixa delimitadora
Fonte: Do próprio autor

- **Fase Estreita:** Uma vez que ocorre a intersecção entre os dois corpos efetivamente, o corpo controlável se torna vermelho, vide Figura 21.

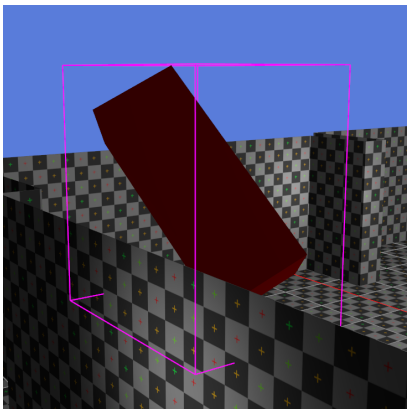


Figura 21 – Intersecção entre os corpos
Fonte: Do próprio autor

A simulação possui uma "camera" centrada no corpo controlável, que pode ser rotacionada e aproximada para melhor visibilidade dos pontos de contato entre os objetos.

Esta simulação permite uma validação visual da corretude e precisão do algoritmo **GJK** na prática, além de demonstrar os resultados corretos das intersecções entre corpos de diferentes tipos.

4.6.2 Dados Numéricos

Além da simulação interativa, uma sequência de testes foi gerada para obter dados estatísticos númericos das verificações de intersecção para cada tipo de corpo, descritos em detalhe na Seção 4.7 deste capítulo. Apenas corpos do mesmo tipo são testados entre si nesta série de testes.

Para cada tipo de primitiva, foram criados 10 corpos:

- Cada corpo possui um tamanho que varia de $0.5m$ a $1.5m$ em cada dimensão;
- Eles são dispostos em um espaço de $2 \times 2 \times 2m$ aleatoriamente;
- Cada vez que um corpo é criado, um eixo de rotação qualquer é definido para ele;
- Cada corpo é rotacionado em seu respectivo eixo num ângulo de $-\pi/2$ a $\pi/2$;
- Este teste é repetido 2000 vezes, cada vez com 10 novos corpos.

Para as cascas convexas, os 10 corpos inicialmente são gerados como polítopos de 25 vértices, e a cada 2000 testes, o número de vértices por corpo aumenta em 25, até chegar em corpos de 1000 vértices.

Cada teste com as mesmas 10 primitivas são executados 10000 vezes, e cada teste com os mesmos 10 poliedros de casca convexa são executados 2000 vezes, para que seja extraída uma margem de tempo entre o ínicio e o fim dos testes, e tirada uma média. Isto foi feito pois cada execução do algoritmo ocorre em uma escala de centenas de nanosegundos, então podem sofrer variações em função de mudanças de processos do sistema operacional em execução, variações de temperatura da máquina e disponibilidade de memória RAM.

Para obtenção de margens de tempo com maior precisão, foi utilizada a estrutura std::chrono::steady_clock, que garante uma medição de tempo monotônica[4] e o tempo entre tiques do relógio é constante (JOSUTTIS, 2012).

Todos os testes são realizados sem uma fase ampla, para garantir que o algoritmo **GJK** é rodado independente das proximidades entre os corpos. Estes testes são realizados em uma máquina com as seguintes configurações:

Processador: Intel Core I7-8700K 3.70 GHz, 6 núcleos, 12 threads

RAM: Kingston DDR4 2666 MHz DIMM 16 GB (2×8 GB)

Armazenamento: Kingston SA400S37480G SSD 480 GB 450 MB/s

Sistema Operacional: Windows 10 Education 64 bit Version 10.0.18363 Build 18363

Os testes foram compilados usando o compilador GNU g++ com sem otimização(*flag* -O0), sem desenrolamento de *loops*, sem paralelismo, sem utilização de instruções SIMD, e sem operações em GPU.

4.7 Resultados

A sequência de testes gerou um conjunto de dados de máximo de iterações percorridas por execução do **GJK** entre pares de corpos. Também foi calculado o número máximo de vezes, em um único teste, que um *simplex* é simplificado para triângulo, aresta ou para ponto. Por fim, extraiu-se a média de tempo, em microssegundos, que a rotina **GJK** leva para ser executada do começo ao fim uma única vez.

Com base nos dados coletados, tem-se o objetivo de verificar e confirmar as três afirmações feitas no Capítulo 3, lembradas na sequência:

1. O laço de repetição principal do algoritmo **GJK**, apesar de **não** possuir um número de iterações pré-definido, executa em uma quantidade de vezes dentro de uma margem constante para cada tipo de forma geométrica;

2. A complexidade de tempo do algoritmo tende a ser constante para formas primitivas e linear para poliedros de casca convexa;

[4]Um relógio é dito monotônico quando uma medição de tempo é igual ou inferior à medição de tempo subsequente.

3. A quantidade de vezes em que o *simplex* formado diminui para um 0-*simplex* (que ele volta a ser um único ponto) é nula, independente da forma geométrica testada, e portanto, efetivamente esta condição não precisa ser testada.

As Subseções 4.7.1 e 4.7.2 apresentam os resultados obtidos para os testes feitos com primitivas e poliedros de cascas convexas, respectivamente.

4.7.1 Primitivas

Os 2000 testes efetuados para conjuntos de 10 primitivas de cada tipo mostram uma oscilação estável entre os valores relativos a tempo de execução e número de iterações dentro do algoritmo **GJK**. As figuras de 22 a 26 apresentam os resultados em diagramas de caixa[5], para auxiliar na visualização de comportamento médio e *outliers*.

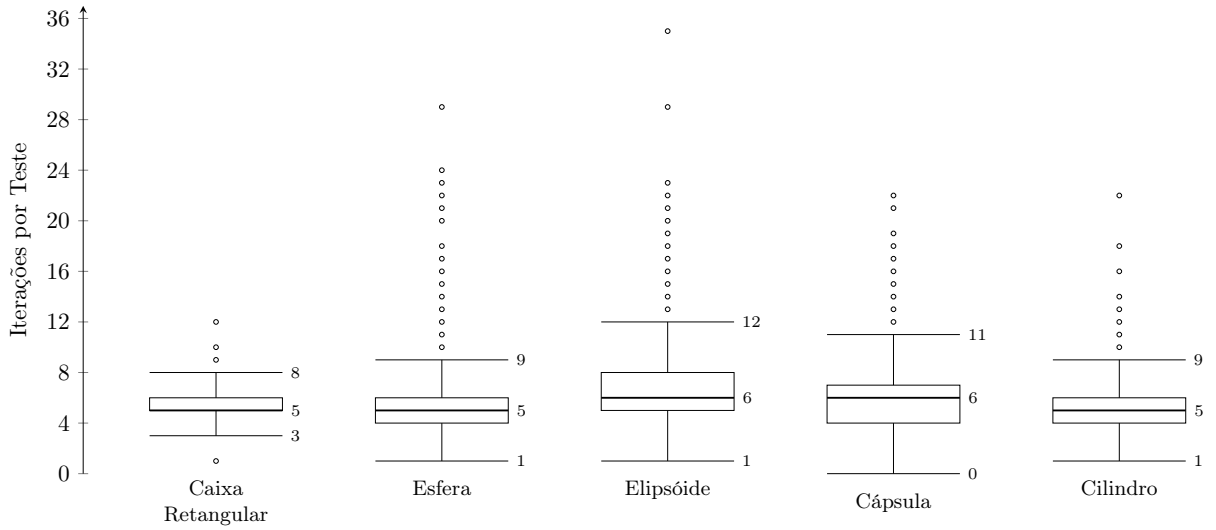


Figura 22 – Número máximo de iterações do laço de repetição do algoritmo por par de corpos

A Figura 22 apresenta o número máximo de iterações dentro do algoritmo **GJK** para cada tipo de primitiva. Observa-se que as medianas tem uma variabilidade, de uma primitiva para outra, de apenas 1 iteração, e os quartis se mantém dentro de uma margem de 4 a 8 iterações.

Estabelecendo uma comparação com o algoritmo **SAT**, segundo (GOTTSCHALK; LIN; MANOCHA, 1996), uma implementação particular para caixas orientadas do algoritmo **SAT** possui um laço de 15 iterações, que deve ser percorrido por completo para confirmar a intersecção entre duas caixas, enquanto a implementação em **GJK** mostra resultados dentro de uma média de 5 a 6 repetições. Outro exemplo é o algoritmo de

[5]Diagramas de caixa, ou *boxplots* são gráficos que apresentam o comportamento de uma dada variável através de quartis, medianas, limites inferior e superior e *outliers* (valores discrepantes) (MANN, 2012).

intersecção proposto por MALAPALLY (2016) para elipsóides, cujos resultados apresentam uma média 12 iterações, e chegando a máximas de até 50 iterações, comparado aos resultados mostrados na Figura 22, que mostram uma faixa média de 5 a 8 iterações para elipsóides, com uma máxima de 35.

A porcentagem de *outliers* para o número de iterações na sequência de testes de cada primitiva ocorreu de acordo com os valores dados na Tabela 2.

Forma Geométrica	% de Outliers
Caixas Retangulares	5.8%
Esferas	3.25%
Elipsóides	5.3%
Cápsulas	2.7%
Cilindros	4.15%

Tabela 2 – Porcentagem de Ocorrência de *Outliers* (Número de Iterações)

Analisando os resultados da Figura 22 e da Tabela 2, conclui-se que o laço de repetição presente no algoritmo **GJK** tende a ser constante, nos testes com geometrias primitivas, conforme afirmado no Capítulo 3, segundo CAMERON (1997).

Outro resultado coletado dos testes foi o número máximo de vezes, dentro de uma execução do algoritmo **GJK**, em que o simplex contido pelo **CSO** foi reduzido em um vértice, isto é, quantas vezes ele foi para um ponto, para uma aresta e para triângulo. As figuras 23 a 25 apresentam estes resultados, respectivamente.

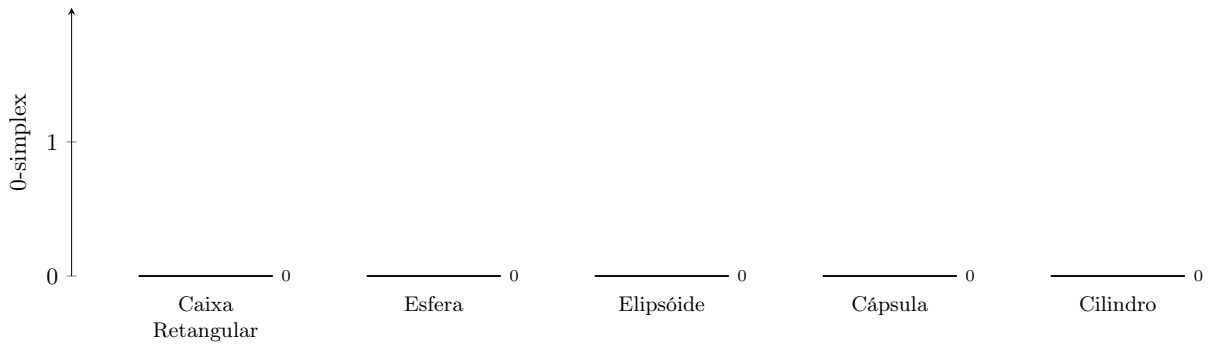


Figura 23 – Número máximo de ocorrências de diminuição do *simplex* para um **ponto** por teste entre par de corpos

A Figura 23 mostra o número de casos em que o *simplex* retorna a um 0-*simplex* (ou seja, apenas um vértice), e mostra que não há ocorrência destes casos, para corpos representados por primitivas.

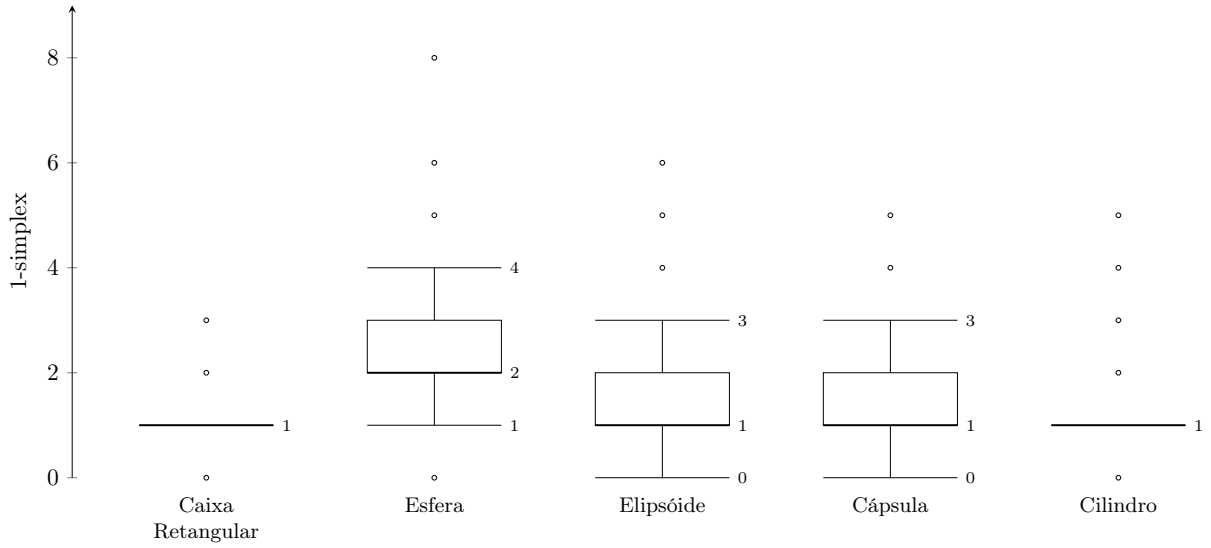


Figura 24 – Número máximo de ocorrências de diminuição do *simplex* para uma **aresta** por teste entre par de corpos

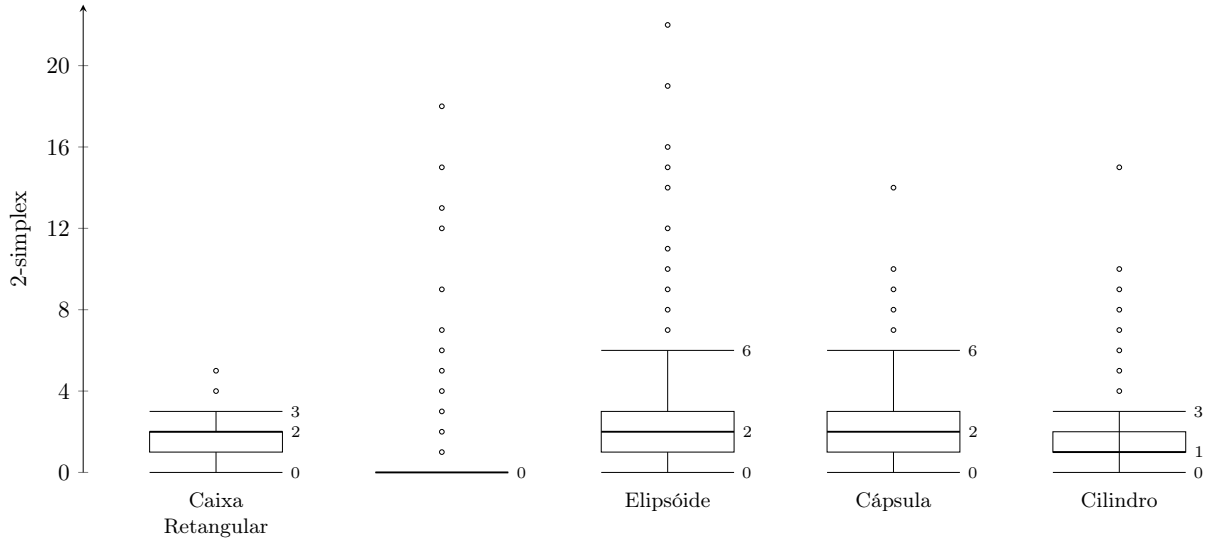


Figura 25 – Número máximo de ocorrências de diminuição do *simplex* para um **triângulo** por teste entre par de corpos

As figuras 24 e 25 mostram os números de casos em que o simplex retorna a um 1-simplex (aresta) e 2-simplex (triângulo), respectivamente. As variabilidades dos resultados vão de 0 a 2, com medianas entre 1 e 2. Estes dados expressam a rapidez com que o algoritmo **GJK** encontra um tetraedro que contem a origem do espaço vetorial ou conclui que o **CSO** não contem este ponto, o que explica porque as iterações do laço de repetição tendem a ocorrer dentro de uma margem constante.

A Tabela 3 mostra a porcentagem de ocorrências de *outliers* para as diminuições do *simplex* para 1 e 2-*simplex*.

Forma Geométrica	% Outliers (1-simplex)	% Outliers (2-simplex)
Caixas Retangulares	18.45%	1.95%
Esferas	1.15%	6.65%
Elipsóides	1.2%	4.6%
Cápsulas	0.55%	1.6%
Cilindros	28.65%	10.3%

Tabela 3 – Porcentagem de Ocorrência de *Outliers* (Redução de Simplex)

As porcentagens mais altas de *outliers* na tabela, no caso de caixas retangulares e cilindros, se dá pela quantidade de vezes em que o algoritmo não mostrou nenhuma ocorrência de redução para 1-*simplex*, que ocorreu para a maioria dos *outliers* destas duas primitivas.

O último resultado coletado dos testes foi o tempo médio de execução do algoritmo **GJK**, em µs apresentado na Figura 26.

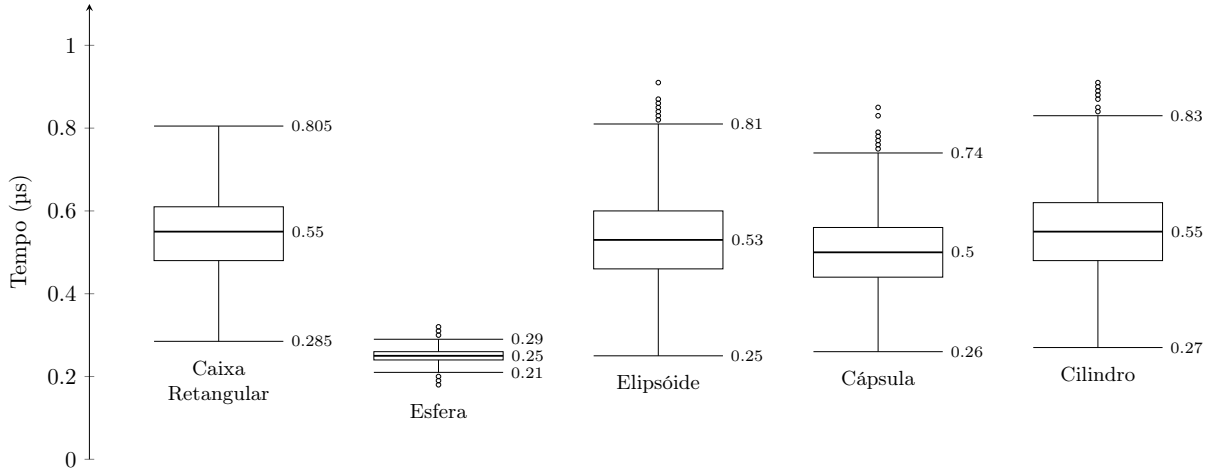


Figura 26 – Tempo médio de execução do algoritmo **GJK** por par de corpos

A execução do algoritmo para todas as primitivas se manteve abaixo de 1 µs, dentro de uma média entre 0.5 e 0.55 µs, com exceção dos resultados dos testes com esferas. Existem dois possíveis motivos para o algoritmo ter esta diferença de performance com esferas:

- A função *Support* da esfera executa menos operações do que as demais, o suficiente para haver uma diferença perceptível de performance (conforme descrito na Tabela 1);
- A rotação de uma esfera não precisa ser considerada para funcionar corretamente com o algoritmo **GJK**, pois sua forma não se altera independente da rotação. Portanto, a função *Support* para esta primitiva não contém rotações, que são aplicadas nas implementações das outras primitivas.

Considerando que rotações quaternianas (utilizadas na implementação das funções *Support*) possuem operações trigonométricas de seno e cosseno, e observando a simplicidade da parametrização da função *Support* para esferas, o tempo de execução do algoritmo **GJK** pode apresentar diferenças sensíveis, particularmente levando em conta a escala de variação de performance, de menos de 1 µs.

A Tabela 4 mostra a porcentagem de ocorrências de *outliers* em relação a tempo de execução do algoritmo para os 5 tipos de primitivas. Observa-se uma ocorrência quase nula destes casos, a despeito da presença de execuções do **GJK** com números mais altos de iterações.

Forma Geométrica	% de Outliers
Caixas Retangulares	0.0%
Esferas	1.0%
Elipsóides	0.85%
Cápsulas	0.65%
Cilindros	0.5%

Tabela 4 – Porcentagem de Ocorrência de *Outliers* (Tempo de Execução)

4.7.2 Poliedros de Casca Convexa

Diferentemente dos testes efetuados com primitivas, foram executados 40000 testes para conjuntos de 10 corpos de casca convexa, cada corpo começando com 25 vértices, e a cada 1000 testes, incrementando em mais 25 vértices por corpo. Esta sequência foi programada para demonstrar o crescimento linear do tempo de execução do algoritmo **GJK** quando executado para testar a intersecção entre formas convexas.

A Figura 27 apresenta lado a lado os diagramas de caixas referentes ao número máximo de iterações do laço de repetição do algoritmo, e os números máximos de redução do *simplex* para um ponto, uma aresta e um triângulo, respectivamente.

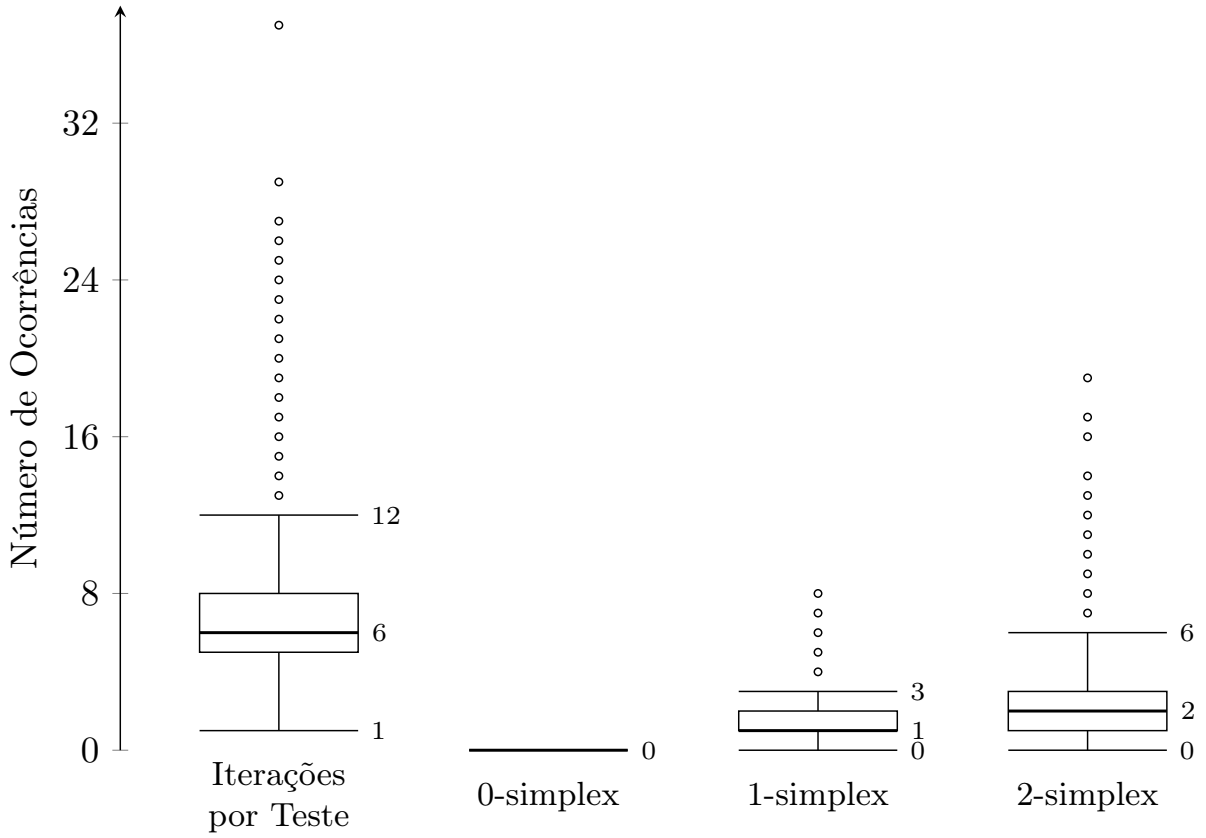


Figura 27 – Iterações do laço de repetição
e casos de redução do simplex

Nota-se no diagrama de número de iterações por teste que o laço de repetição principal do algoritmo **GJK** continua apresentando um comportamento tendendo a constante, dentro de uma faixa de variação quase idêntica à do diagrama de número de iterações observado nos testes de elipsóides, na Figura 22. Os diagramas de número de reduções do *simplex* para 0, 1 e 2-*simplex* também replicam os resultados vistos nos testes feitos com primitivas, observados nas Figuras 23, 24 e 25. Dois aspectos importantes são observados:

- Assim como para formas paramétricas, os testes com cascas convexas não tiveram nenhuma ocorrência de redução do simplex para um único ponto, e mais uma vez não é necessário testar estes casos na função *NearestSimplex*;
- Independente da parametrização da forma geométrica ou complexidade da malha convexa, o laço de repetição do **GJK** e a quantidade de vezes em que a função *NearestSimplex* remove pontos do simplex tende a variar dentro de uma margem pequena e similar para todos os casos.

A Tabela 5 explicita a porcentagem de ocorrências de *outliers* para cada métrica da Figura 27. Como nos testes feitos com primitivas, a presença destes casos é sensivelmente

reduzida, reforçando a tendência do **GJK** de rodar dentro de uma quantidade constante de vezes.

Métrica	% de Outliers
Número de Iterações	3.48%
Redução para 0-simplex	0.0%
Redução para 1-simplex	0.99%
Redução para 2-simplex	2.79%

Tabela 5 – Porcentagem de Ocorrência de *Outliers* (Número de Iterações e redução do *simplex*)

Os resultados obtidos de tempo médio de execução para cada par de corpos convexos são mostrados pela Figura 28. Apesar de todos resultados de quantidade de iterações do algoritmo apresentarem números dentro de uma margem constante, o tempo de execução aumenta linearmente conforme os corpos de colisão adquirem mais vértices.

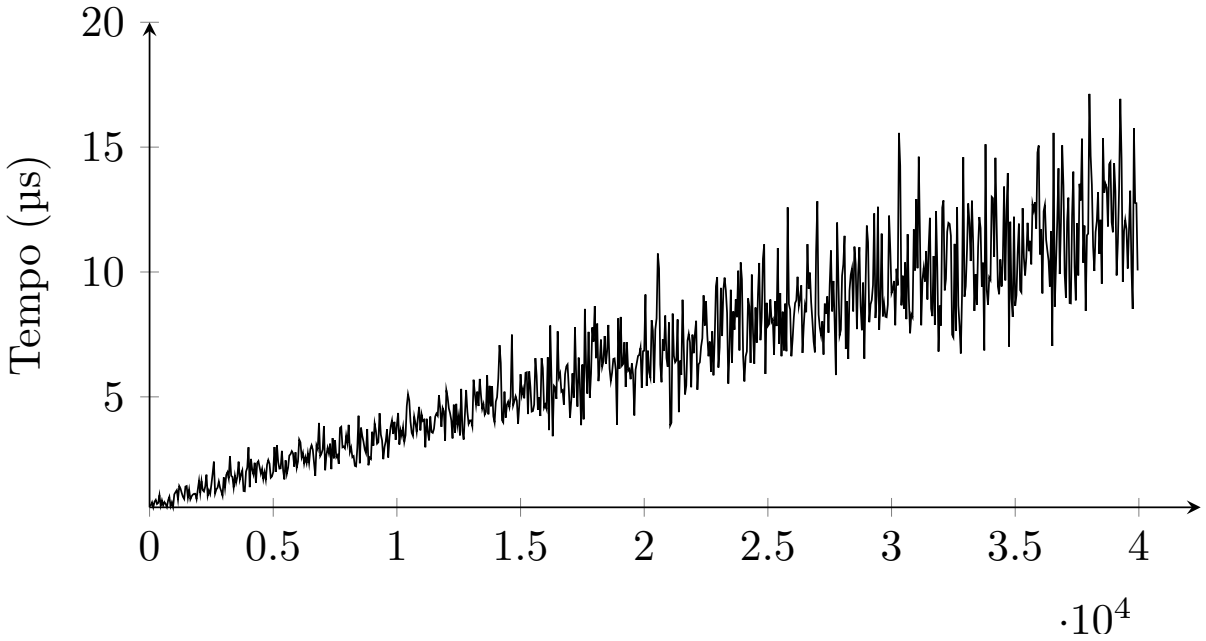


Figura 28 – Tempo médio de execução do algoritmo **GJK** por par de corpos

Além do crescimento linear do tempo de execução, observa-se também uma variação maior entre os mínimos e máximos ao longo dos testes. Estes dois fenômenos estão diretamente relacionados à função *Support*.

Conforme apresentado no Capítulo 3, a implementação da função *Support* para corpos de casca convexa é um algoritmo de complexidade de tempo de ordem $O(N)$, sendo N o número de pontos que representam o poliedro. Cada vez que o algoritmo **GJK** faz uma chamada desta função, o tempo de processamento aumenta com o laço de

repetição de busca pelo ponto mais distante em uma direção $\vec{d}$, e uma pequena diferença na quantidade de chamadas da função *Support*, portanto, provoca uma diferença substancial na performance do teste de intersecção como um todo.

Por conta dos fatores apresentados, na literatura e nos dados obtidos nos testes efetuados, diversas melhorias propostas para o algoritmo **GJK** envolvem métodos de busca mais rápidos na função *Support* (MONTANARI; PETRINIC; BARBIERI, 2016), uma vez que a rotina principal do **GJK** e a função *NearestSimplex* operam em ordem constante e dependem de funções matemáticas computacionalmente pouco custosas, pois não executam operações matriciais, trigonométricas, divisões ou raízes.

5 Conclusão e Trabalhos Futuros

Este capítulo apresenta as conclusões sobre as análises realizadas no Capítulo 3 e os resultados obtidos dos testes no Capítulo 4. Em seguida, será descorrido sobre possíveis desdobramentos da biblioteca criada em trabalhos futuros.

5.1 Conclusão sobre Análises e Resultados de Testes

Os algoritmos **SAT** e **GJK** de intersecção entre corpos rígidos convexos são utilizados extensamente entre bibliotecas, *frameworks* e *engines* de física para simulações interativas (COUMANS, 2012; COUMANS, 2013; CATTO, 2013; GREGORIUS, 2013; OLVÅNG, 2010), além de computação física científica (MALAPALLY, 2016; HE et al., 2019).

Uma análise inicial sobre a sequência destes algoritmos, acrescida de resultados relatados na literatura, aponta algumas vantagens da abordagem do **SAT** em relação ao **GJK**:

1. O algoritmo **SAT** usa conceitos elementares de geometria (eixos de separação), o que torna o método implementável sem que haja um código ou pseudocódigo de referência;

2. O laço de repetição central do **SAT**, que determina a complexidade de tempo do algoritmo, é paralelizável. Como cada iteração do algoritmo ocorre independentemente, pode-se utilizar recursos de paralelismo de CPU e GPU para executar o algoritmo efetivamente em uma complexidade de tempo menor;

3. Segundo HORNUS (2017), comparações de performance entre o **SAT** e o **GJK** apontam que malhas com baixa densidade de vértices, tais como caixas retangulares, são testadas mais rapidamente pelo algoritmo **SAT**.

O algoritmo **GJK**, por sua vez, oferece vantagens quanto a complexidade assintótica de tempo e generalização da implementação:

1. Como visto no Capítulo 3, o **GJK** tem complexidade de tempo linear, tanto em $\mathbb{R}^2$ quanto em $\mathbb{R}^3$;

2. O **GJK** pode ser utilizado com poliedros representados apenas por conjuntos de vértices, mantendo tempo de execução de ordem linear. O **SAT** requer estruturas de

dados específicas de representação de arestas/faces e suas normais para conseguir operar em complexidade linear, se não ele opera em $O(N^2)$ em $\mathbb{R}^2$ e $O(N^3)$ em $\mathbb{R}^3$;

3. As diferentes implementações da função *Support* para formas paramétricas e polítopos convexos permite que haja uma única função **GJK** para lidar com tipos de geometrias distintas, e associar a função *Support* diretamente a cada corpo de colisão, tornando o algoritmo organizado e modular;

4. A implementação da função **GJK** e da função *NearestSimplex* para geometrias de diferentes tipos é a mesma em todos os casos. O algoritmo **SAT** requer implementações particulares para lidar com formas curvas (em certos casos, menos triviais, como elipsóides) (MALAPALLY, 2016) ou para operar mais rapidamente sobre caixas retangulares (GOTTSCHALK; LIN; MANOCHA, 1996).

Esta análise, feita preliminarmente em cima das descrições e dos pseudocódigos dos dois algoritmos, aliada às motivações, dificuldades e resultados reportados na literatura, sugere uma vantagem do algoritmo **GJK** em relação ao **SAT**, levando em conta que os dois métodos foram analisados em suas formas iniciais, desconsiderando otimizações ou variações de implementações sugeridas posteriormente.

Conforme a série de testes realizadas, no Capítulo 4, os dados coletados validam afirmações colocadas no Capítulo 3 e na literatura, além de contribuírem com informações importantes sobre o comportamento do **GJK**:

1. O laço de repetição central do algoritmo tende a ocorrer dentro de uma margem constante, validando a análise assintótica feita anteriormente, e mostrando em dados númericos o comportamento reportado na literatura (CAMERON, 1997);

2. Os resultados evidenciam a tendência de complexidade de tempo constante do **GJK** para lidar com formas geométricas primitivas, e linear para poliedros de casca convexa, devido à diferença de complexidade de tempo da função *Support* para cada tipo de geometria;

3. O número de ocorrências de redução do *simplex* para um único ponto (ou 0-*simplex*) foi 0 em todos os testes, nos 2000 testes para cada 10 corpos de cada um dos 5 tipos de primitiva implementados, e nos 40000 testes efetuados sobre corpos convexos contendo de 25 a 1000 vértices. Isto implica que todos os testes dentro da função *NearestSimplex* que preveem estes casos são desnecessários;

4. Os testes efetuados entre caixas retangulares apontam que o número de iterações do algoritmo **GJK** para estas primitivas ocorre dentro de uma margem de 4 a 6 repetições, com o pior caso chegando a 12 iterações. Estes números são menores do

que a implementação particular de caixas retangulares do **SAT** propostas segundo GOTTSCHALK, LIN e MANOCHA (1996), que para confirmar a intersecção entre duas caixas, requer iterar todos os 15 possíveis eixos de separação;

5. Os testes entre elipsóides tendem a ter de 5 a 8 iterações, com ocorrências no máximo de 36 iterações, números menores do que os resultados segundo MALAPALLY (2016), obtidos da implementação particular do algoritmo **SAT**, que apresentam uma média de 12 iterações por teste, e chegando a máximas de 50 iterações.

Em suma, dentro do contexto analisado para implementar a biblioteca proposta, diante dos resultados obtidos dos testes, e considerando as conclusões reportadas pela bibliografia estudada, observa-se a robustez do algoritmo **GJK** quando comparado ao **SAT**, em relação a sua versatilidade, escalabilidade, performance, e possibilidade de melhoria em desempenho (CAMERON, 1997; MONTANARI; PETRINIC; BARBIERI, 2016; OLVÅNG, 2010).

5.2 Trabalhos Futuros

A biblioteca de intersecção desenvolvida serve como módulo para uma ferramenta de física para simulações interativas. Podem ser desenvolvidas técnicas de tratamento de colisão, particionamento espacial, particionamento de formas côncavas em corpos convexos, e detecção contínua de colisão em conjunto com esta ferramenta, para comporem uma biblioteca completa de física de corpos rígidos.

Outro desdobramento é o refinamento organizacional e de performance da biblioteca. Uma reestruturação do código deve ser feita para melhor legibilidade e manutebilidade, e é necessário um estudo e desenvolvimento aprofundado das otimizações propostas por outros autores, a fim de otimizar a eficiência da biblioteca, seja através de estratégias de algoritmos e estruturas de dados que acelerem buscas no código (CAMERON, 1997), ou fazendo uso de recursos computacionais modernos, paralelismo ou **SIMD**[1], por exemplo (COUMANS, 2013).

A biblioteca foi criada com a finalidade de descrever intersecção entre corpos rígidos. Uma biblioteca para lidar com corpos flexíveis/deformáveis pode ser desenvolvida para trabalhar em conjunto com a ferramenta já implementada, ou esta biblioteca pode ser incrementada para lidar com estes casos.

Por fim, segundo OLVANG (2010), o próprio algoritmo **GJK** possui diversas variações que podem ser estudadas para encontrar a solução mais compatível com a finalidade da biblioteca desenvolvida neste trabalho. Também pode-se desenvolver uma

[1]Sigla para instruções de máquina que processam múltiplos dados simultaneamente (*Single Instruction, Multiple Data*).

implementação híbrida entre o **GJK** e o **SAT**, conforme proposto por GREGORIUS (2015). Além disso, outras técnicas de colisão entre formas geométricas diversas são propostas periodicamente, e que potencialmente podem substituir por completo o algoritmo **GJK** implementado (ou outros métodos de detecção de intersecção entre corpos), desde técnicas baseadas em *Machine Learning* (HOLDEN et al., 2019) a algoritmos baseados em *Signed Distance Fields* (MACKLIN et al., 2020).

Referências

ALEARDI, L. C. et al. Succinct representation of triangulations with a boundary. In: *9th Workshop on Algorithms and Data Structures*. Waterloo, Canada: [s.n.], 2005. p. 134–135. Citado na página 28.

BAEK, J. et al. Visualizing Quaternion Multiplication. *IEEE Access*, v. 5, p. 8948–8955, 2017. ISSN 21693536. Citado na página 20.

BERG, M. de et al. *Computational Geometry: Algorithms and Applications*. 2nd editio. ed. Berlin; Heidelberg; New York; Barcelona; Hong Kong; London; Milan; Paris; Singapore; Tokyo: Springer-Verlag, 1997. ISSN 14337851. ISBN 9783540779735. Citado na página 15.

BROCHU, T.; EDWARDS, E.; BRIDSON, R. Efficient geometrically exact continuous collision detection. *ACM Transactions on Graphics*, v. 31, n. 4, p. 1–7, 2012. ISSN 07300301. Citado na página 13.

CAMERON, S. Enhancing GJK: Computing Minimum and Penetration Distances between Convex Polyhedra. *International Conference on Robotics & Automation, Albuquerque*, p. 0–5, 1997. Citado 5 vezes nas páginas 30, 39, 43, 62 e 63.

CATTO, E. Physics for Game Programmers; Continuous Collision. In: *Game Developer Conference*. [S.l.: s.n.], 2013. Citado 2 vezes nas páginas 48 e 61.

CHEN, J. Computational Geometry: Methods and Applications. 1996. Citado na página 15.

CORMEN, T. H. et al. *Introduction to Algorithms*. 3rd editio. ed. Cambridge, Massachusetts: The MIT Press, 2009. Citado na página 23.

COUMANS, E. Bullet 2.80 Physics SDK Manual. 2012. Citado 2 vezes nas páginas 46 e 61.

COUMANS, E. GPU Rigid Body Simulation. *Game Developer Conference*, 2013. Citado 6 vezes nas páginas 13, 14, 43, 44, 61 e 63.

ERICSON, C. *The Gilbert-Johnson-Keerthi algorithm*. [S.l.]: Elsevier, 2005. Citado na página 30.

GILBERT, E. G.; JOHNSON, D. W.; KEERTHI, S. S. A Fast Procedure for Computing the Distance Between Complex Objects in Three-Dimensional Space. *IEEE Journal of Robotics and Automation*, v. 4, n. 2, 1988. Citado na página 48.

GOTTSCHALK, S.; LIN, M. C.; MANOCHA, D. OBB tree: A hierarchical structure for rapid interference detection. *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1996*, p. 171–180, 1996. Citado 3 vezes nas páginas 28, 52 e 62.

GREGORIUS, D. The Separating Axis Test between Convex Polyhedra. In: . [S.l.: s.n.], 2013. Citado 6 vezes nas páginas 25, 26, 27, 28, 44 e 61.

GREGORIUS, D. Robust Contact Creation for Physics Simulations. In: *Game Developer Conference*. San Francisco, CA: [s.n.], 2015. Citado 2 vezes nas páginas 14 e 43.

HE, H. et al. Simulation of realistic particles with bullet physics engine. *E3S Web of Conferences*, v. 92, p. 1–5, 2019. ISSN 22671242. Citado na página 61.

HOLDEN, D. et al. Subspace neural physics: Fast data-driven interactive simulation. *Proceedings - SCA 2019: ACM SIGGRAPH / Eurographics Symposium on Computer Animation*, 2019. Citado na página 64.

HORNUS, S. Detecting the intersection of two convex shapes by searching on the 2-sphere. *CAD Computer Aided Design*, v. 90, p. 71–83, 2017. ISSN 00104485. Citado na página 44.

HOUSE, D. H. Winged-Edge Data Structure. 2011. Citado na página 44.

HUGHES, J. F. et al. *Computer Graphics: Principles and Practice*. 3rd editio. ed. Upper Saddle River, New Jersey: Pearson Education, Inc., 2014. ISBN 9780321399526. Citado na página 20.

JOSUTTIS, N. M. *The C++ Standard Library: A Tutorial and Reference*. 2nd. ed. [S.l.]: Addison-Wesley Professional, 2012. ISBN 0321623215. Citado na página 51.

KNUTH, D. E. *The Art of Computer Programming: Fundamental Algorithms*. 3rd editio. ed. Reading, Massachusetts; Harlow, England; Menlo Park, California; Berkeley, California; Don Mills, Ontario; Sydney; Bonn; Amsterdam; Tokyo; Mexico City: Addison Wesley Longman, 1997. v. 1. 650 p. ISBN 0-201-89683-4. Citado na página 23.

MACKLIN, M. et al. Local Optimization for Robust Signed Distance Field Collision. *Proc. ACM Comput. Graph. Interact. Tech. 3, 1, Article*, v. 8, n. 1, p. 17, 2020. Disponível em: <https://doi.org/10.1145/3384538>. Citado 2 vezes nas páginas 13 e 64.

MALAPALLY, N. *Ellipsoid collision detection support for pe: ellipsoid-ellipsoid and ellipsoid-plane collisions*. Dissertação (Mestrado) — Friedrich-Alexander-Universität Erlangen-Nürnberg, 2016. Citado 3 vezes nas páginas 44, 61 e 62.

MANN, P. *Introductory Statistics, 8th Edition*. John Wiley and Sons, Incorporated, 2012. ISBN 9781118473986. Disponível em: <https://books.google.com.br/books?id=RTYcAAAAQBAJ>. Citado na página 52.

MONTANARI, M.; PETRINIC, N.; BARBIERI, E. Improving the GJK algorithm for faster and more reliable distance queries between convex objects. *ACM Transactions on Graphics*, 2016. ISSN 07300301. Citado 13 vezes nas páginas 30, 33, 35, 36, 37, 38, 39, 40, 41, 42, 43, 59 e 63.

OLVÅNG, L. Real-time Collision Detection with Implicit Objects. *ACM Trans. Graph. ()*, v. 32, n. 4, p. 1, 2010. Disponível em: <http://dl.acm.org/citation.cfm?doid=2461912.2461960>. Citado 6 vezes nas páginas 32, 34, 40, 49, 61 e 63.

O'ROURKE, J. *Computational Geometry in C*. 2nd editio. ed. [S.l.]: Cambridge University Press, 1997. Citado na página 18.

SIPSER, M. *Introduction to the Theory of Computation Third Edition*. 3rd editio. ed. [S.l.]: Cengage Learning, 2013. 275 p. ISBN 9781133187790. Citado na página 23.

ZHANG, X.; LEE, M.; KIM, Y. J. Interactive continuous collision detection for non-convex polyhedra. *Visual Computer*, v. 22, n. 9-11, p. 749–760, 2006. ISSN 01782789. Citado na página 48.

A Apêndice

A.1 Operações Matemáticas

Distância Euclidiana[1] Seja $\overline{AB}$ uma aresta que conecta os pontos $A = (a_1, a_2, a_3)$ e $B = (b_1, b_2, b_3)$. A distância euclidiana ou comprimento de $\overline{AB}$ pode ser calculado por

$$\|\overline{AB}\| = d(A, B) = \sqrt{(b_1 - a_1)^2 + (b_2 - a_2)^2 + (b_3 - a_3)^2}$$

O resultado desta operação é um número pertencente a $\mathbb{R}_+$. A distância euclidiana entre dois pontos/vértices é comutativa[2] e distributiva[3].

Produto Escalar[1]**:** Sejam $\vec{A}$ e $\vec{B}$ vetores, então o produto escalar entre eles é obtido por

$$\vec{A} \cdot \vec{B} = a_1 b_1 + a_2 b_2 + a_3 b_3$$

O resultado desta operação é um número em $\mathbb{R}$. O produto escalar entre dois vetores é distributivo, associativo[4] e comutativo.

Produto Vetorial[1]**:** Sejam $\vec{A}$ e $\vec{B}$ dois vetores, então o produto vetorial entre eles é obtido por

$$\vec{A} \times \vec{B} = \hat{\imath}(a_2 b_3 - a_3 b_2) + \hat{\jmath}(a_3 b_1 - a_1 b_3) + \hat{k}(a_1 b_2 - a_2 b_1),$$
$$\{\hat{\imath}, \hat{\jmath}, \hat{k}\} \text{ é a base canônica de } \mathbb{R}^3.$$

O resultado desta operação é um vetor $\vec{C}$, $\vec{C} \perp \vec{A}, \vec{B}$, e sua direção é orientada pela convenção da regra da mão direita[5]. O produto vetorial é distributivo, associativo e **anticomutativo**[6].

---

[1] Em $\mathbb{R}^3$.

[2] **Comutatividade:** Seja $\#$ uma operação sobre um conjunto S, $\#$ é comutativa se $x \# y = y \# x, \forall\ x, y \in S$.

[3] **Distributividade:** Sejam $\$, \#$ duas operações sobre um conjunto S, $\$$ é distributiva em relação a $\#$ se $x\ \$\ (y \# z) = (x\ \$\ y) \# (x\ \$\ z)\ \wedge\ (x \# y)\ \$\ z = (x\ \$\ z) \# (y\ \$\ z), \forall\ x, y, z \in S$.

[4] **Associatividade:** Seja $\#$ uma operação sobre um conjunto S, $\#$ é associativa se $x \# (y \# z) = (x \# y) \# z, \forall\ x, y, z \in S$.

[5] A regra da mão direita serve para descrever os eixos x, y e z utilizando os dedos polegar, indicador e médio, respectivamente, e no caso do Produto Vetorial, a multiplicação de um vetor $\vec{A}$ (polegar) por um vetor $\vec{B}$ (indicador) resulta em um vetor $\vec{C}$ (médio) ortogonal a eles.

[6] $\vec{A} \times \vec{B} = -\vec{B} \times \vec{A}$.

A.2 Enumeradores

```
enum shapeType {
    BOX_BODY        = 1 << 0, // Bounding Box 2d/3d
    BALL_BODY       = 1 << 1, // Ball 2d/3d
    ELLIPSOID_BODY  = 1 << 2, // Ellisoid 2d/3d
    CYLINDER_BODY   = 1 << 3, // 3d Only
    CAPSULE_BODY    = 1 << 4, // Capsule 2d/3d
    CONVEX_HULL     = 1 << 5, // Convex body 2d/3d
};

enum behaviorType {
    STATIC    = 1 << 0, // Bodies don't move at all
    KINEMATIC = 1 << 1, // Bodies may be animated, aren't moved by physics
    DYNAMIC   = 1 << 2, // Bodies translate but don't rotate
    RIGID     = 1 << 3  // Bodies both translate and rotate
};
```

A.3 Estruturas de Dados

```
struct v3 {
    float x,y,z;
};

// NOTE: For quaternions
struct v4 {
    float w; // Declaration of real part BEFORE xyz
    v3 vec;
};

struct box3d {
    v3 radius;
    v3 min, max;
};

struct ball3d {
    float radius; // No need for min and max. They don't change
};
```

```
struct ellipsoid3d {
    v3 radius;
    v3 min, max;
};

struct cylinder3d {
    float length;
    float radius;
    v3 min, max;
};

struct capsule3d {
    float length; // length does not account for radius
    float radius;
    v3 min, max;
};

struct convex3d {
    std::vector<v3> vertices;
    v3 min, max;
};

struct colBody3d {
    shapeType shape;      // Type of body to cast to body pointer
    behaviorType behavior;// Influences transforms

    void *attributes;     // Body pointer
    v3 min, max;          // Updated Min and Max of body's AABB
    v3 bodyMin, bodyMax;  // Original min and max of body

    v3 pos;               // Object position in meters
    v4 rotation;          // quaternion rotation of collider

    uint64_t id;          // Id of the body owner

    v3 (*support)(colBody3d, v3);// Support function (GJK)

    bool insideAABB;      // AABB collision
    bool intersect;       // Intersection
```

```
};

struct simulation {
    std::vector<colBody3d *> bodies3d;
    int bodyCount3d() { return bodies3d.size(); }
};
```

A.4 Operadores e Funções Vetoriais

```
// vector sum
v3 operator + (const v3 &a, const v3 &b) {
    return (v3) {a.x + b.x, a.y + b.y, a.z + b.z};
}

// vector difference
v3 operator - (const v3 &a, const v3 &b) {
    return (v3) {a.x - b.x, a.y - b.y, a.z - b.z};
}

// scalar product vector * number
v3 operator * (const v3 &a, const float &b) {
    return (v3) {a.x * b, a.y * b, a.z * b};
}

// scalar product number * vector
v3 operator * (const float &b, const v3 &a) {
    return (v3) {a.x * b, a.y * b, a.z * b};
}

// quaternion multiplication
v4 operator * (const v4 &a, const v4 &b) {
    // q = (s, v3)
    float sA = a.w;
    v3 vA = a.vec;
    float sB = b.w;
    v3 vB = b.vec;

    // q v3
    v3 qV3 = (vB * sA) + (vA * sB);
```

```
    qV3 = qV3 + cross(vA,vB);

    return (v4){(sA*sB) - dot(vA, vB), qV3};
}

// dot product
float dot(v3 a, v3 b) {
    return a.x*b.x + a.y*b.y + a.z*b.z;
}

// cross product
v3 cross(v3 a, v3 b) {
    return (v3) {
        (a.y*b.z - a.z*b.y),
        (a.z*b.x - a.x*b.z),
        (a.x*b.y - a.y*b.x)
    };
}

// vector length
float length(v3 a) {
    return sqrt(dot(a,a));
}

// vector normalization
v3 normalize(v3 a) {
    float norm = length(a);
    return (v3) {
        a.x / norm,
        a.y / norm,
        a.z / norm
    };
}
```

A.5 Funções *Support* para GJK

```
v3 support_AAEllipsoidGJK(colBody3d a, v3 direction) {
    v3 d = normalize(direction);
```

```
    v3 radius = ((ellipsoid3d *)a.attributes)->radius;

    v3 vector = {
        (radius.x*radius.x)*d.x,
        (radius.y*radius.y)*d.y,
        (radius.z*radius.z)*d.z
    };

    float norm = length((v3){radius.x*d.x,
                             radius.y*d.y,
                             radius.z*d.z});
    vector.x /= norm;
    vector.y /= norm;
    vector.z /= norm;

    return vector;
}

v3 support_OEllipsoidGJK(colBody3d a, v3 direction) {
    v4 rot = a.rotation;
    v4 iRot = {rot.w, invert(rot.vec)};
    v4 qDir = {0, direction};
    v3 rotDir = (iRot * (qDir * rot)).vec;
    v3 support = support_AAEllipsoidGJK(a, rotDir);

    qDir = (v4){0, support};
    v3 unRotated = (rot * (qDir * iRot)).vec;

    v3 suppTransf = a.pos + unRotated;
    return suppTransf;
}

v3 support_SegmentGJK(float length, v3 direction) {
    v3 d = direction;
    return (v3){0, 0, copysign(length, d.z)};
}

v3 support_DiscGJK(float radius, v3 direction) {
    v3 d = normalize(direction);
```

```
    d = (v3){d.x, d.y, 0.f};
    d = normalize(d);
    return (d * radius);
}

v3 support_SphereGJK(float radius, v3 direction) {
    return (normalize(direction) * radius);
}

v3 support_OSphereGJK(colBody3d a, v3 direction) {
    return a.pos + support_SphereGJK(((ball3d *)a.attributes)->radius,
                                     direction);
}

v3 support_AABoxGJK(colBody3d a, v3 direction) {
    v3 d = direction;
    v3 radius = ((box3d *)a.attributes)->radius;

    return (v3){copysign(radius.x, d.x),
                copysign(radius.y, d.y),
                copysign(radius.z, d.z)};
}

v3 support_OBoxGJK(colBody3d a, v3 direction) {
    v4 rot = a.rotation;
    v4 iRot = {rot.w, invert(rot.vec)};
    v4 qDir = {0, direction};
    v3 rotDir = (iRot * (qDir * rot)).vec;
    v3 support = support_AABoxGJK(a, rotDir);

    qDir = (v4){0, support};
    v3 unRotated = (rot * (qDir * iRot)).vec;

    v3 suppTransf = a.pos + unRotated;
    return suppTransf;
}

v3 support_AACapsuleGJK(colBody3d a, v3 direction) {
    float height = ((capsule3d *)a.attributes)->length/2.f;
```

```
    float radius = ((capsule3d *)a.attributes)->radius;

    return support_SegmentGJK(height, direction) +
            support_SphereGJK(radius, direction);
}

v3 support_OCapsuleGJK(colBody3d a, v3 direction) {
    v4 rot = a.rotation;
    v4 iRot = {rot.w, invert(rot.vec)};
    v4 qDir = {0, direction};
    v3 rotDir = (iRot * (qDir * rot)).vec;
    v3 support = support_AACapsuleGJK(a, rotDir);

    qDir = (v4){0, support};
    v3 unRotated = (rot * (qDir * iRot)).vec;

    v3 suppTransf = a.pos + unRotated;
    return suppTransf;
}

v3 support_AACylinderGJK(colBody3d a, v3 direction) {
    float height = ((cylinder3d *)a.attributes)->length/2.f;
    float radius = ((cylinder3d *)a.attributes)->radius;
    return support_SegmentGJK(height, direction) +
           support_DiscGJK(radius, direction);
}

v3 support_OCylinderGJK(colBody3d a, v3 direction) {
    v4 rot = a.rotation;
    v4 iRot = {rot.w, invert(rot.vec)};
    v4 qDir = {0, direction};
    v3 rotDir = (iRot * (qDir * rot)).vec;
    v3 support = support_AACylinderGJK(a, rotDir);

    qDir = (v4){0, support};
    v3 unRotated = (rot * (qDir * iRot)).vec;

    v3 suppTransf = a.pos + unRotated;
    return suppTransf;
```

```
}

v3 support_AAConvexGJK(colBody3d a, v3 direction) {
    convex3d *shape = ((convex3d *)a.attributes);
    int numPoints = shape.size();
    float dotVal = -FLT_MAX;
    int index = 0;
    for(int i = 0; i < numPoints; ++i) {
        float curDot = dot(shape.vertices[i], direction);
        if(curDot > dotVal) {
            dotVal = curDot;
            index = i;
        }
    }
    return shape.vertices[index];
}

v3 support_OConvexGJK(colBody3d a, v3 direction) {
    v4 rot = a.rotation;
    v4 iRot = {rot.w, invert(rot.vec)};
    v4 qDir = {0, direction};
    v3 rotDir = (iRot * (qDir * rot)).vec;
    v3 support = support_AAConvexGJK(a, rotDir);

    qDir = (v4){0, support};
    v3 unRotated = (rot * (qDir * iRot)).vec;

    v3 suppTransf = a.pos + unRotated;
    return suppTransf;
}
```

A.6 Funções de Colisor e Simulação

```
// colBody3d creator
colBody3d createColBody3d(uint64_t id,
                          shapeType shape,
                          behaviorType behavior,
                          void *body,
                          size_t bodySize,
```

```
                        float mass,
                        v3 pos,
                        float maxLinearSpeed,
                        float maxAngularSpeed,
                        float bounciness,
                        float stFriction,
                        float dynFriction) {

    colBody3d result;
    result.id = id;
    result.shape = shape;
    result.behavior = behavior;
    result.pos = pos;
    result.rotation = (v4){1,(v3){0,0,0}};
    result.attributes = NULL;
    result.intersect = false;

    switch(shape) {
        case BOX_BODY: {
            result.attributes = (box3d *)malloc(sizeof(box3d));
            memcpy(result.attributes, body, sizeof(box3d));
            result.min = result.bodyMin = (v3)((box3d *)body)->min;
            result.max = result.bodyMax = (v3)((box3d *)body)->max;
            result.support = &support_OBoxGJK;
        } break;
        case BALL_BODY: {
            result.attributes = (ball3d *)malloc(sizeof(ball3d));
            memcpy(result.attributes, body, sizeof(ball3d));

            // NOTE: Ball2d and Ball3d structs do not contain min and max vectors
            result.min = result.bodyMin = (v3){-((ball3d *)body)->radius,
                                               -((ball3d *)body)->radius,
                                               -((ball3d *)body)->radius};
            result.max = result.bodyMax = (v3){ ((ball3d *)body)->radius,
                                                ((ball3d *)body)->radius,
                                                ((ball3d *)body)->radius};
            result.support = &support_OSphereGJK;
        } break;
        case ELLIPSOID_BODY: {
```

```
            result.attributes = (ellipsoid3d *)malloc(sizeof(ellipsoid3d));
            memcpy(result.attributes, body, sizeof(ellipsoid3d));
            result.min = result.bodyMin = ((ellipsoid3d *)body)->min;
            result.max = result.bodyMax = ((ellipsoid3d *)body)->max;
            result.support = &support_OEllipsoidGJK;
        } break;
        case CYLINDER_BODY: {
            result.attributes = (cylinder3d *)malloc(sizeof(cylinder3d));
            memcpy(result.attributes, body, sizeof(cylinder3d));
            result.min = result.bodyMin = ((cylinder3d *)body)->min;
            result.max = result.bodyMax = ((cylinder3d *)body)->max;
            result.support = &support_OCylinderGJK;
        } break;
        case CAPSULE_BODY: {
            result.attributes = (capsule3d *)malloc(sizeof(capsule3d));
            memcpy(result.attributes, body, sizeof(capsule3d));
            result.min = result.bodyMin = ((capsule3d *)body)->min;
            result.max = result.bodyMax = ((capsule3d *)body)->max;
            result.support = &support_OCapsuleGJK;
        } break;
        case CONVEX_HULL: {
            result.attributes = (convex3d *)malloc(bodySize);
            memcpy(result.attributes, body, bodySize);
            minMax(&result.bodyMin,
                   &result.bodyMax,
                   (convex3d *)result.attributes.vertices,
                   (convex3d *)result.attributes.numVertices);
            result.min = result.bodyMin;
            result.max = result.bodyMax;
            result.support = &support_OConvexGJK;
        } break;
    }

    return result;

}
```

```
// colBody3d Axis-Aligned Bounding Box update
void updateColBody3dBounds(colBody3d *colBody) {

    v3 min = colBody->bodyMin;
    v3 max = colBody->bodyMax;

    if(colBody->shape & BOX_BODY) {

        v3 points[8] = {(v3){min.x,min.y,min.z},
                        (v3){max.x,min.y,min.z},
                        (v3){min.x,max.y,min.z},
                        (v3){max.x,max.y,min.z},
                        (v3){min.x,min.y,max.z},
                        (v3){max.x,min.y,max.z},
                        (v3){min.x,max.y,max.z},
                        (v3){max.x,max.y,max.z}};

        v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};
        for(int i = 0; i < 8; ++i) {
            v4 qpoint = (v4){ 0, points[i] };
            points[i] = (colBody->rotation * (qpoint * iq)).vec;
        }

        minMax(&colBody->min, &colBody->max, points, 8);
        return;

    } else if(colBody->shape & BALL_BODY) {

        return; // No change for Ball body

    } else if(colBody->shape & ELLIPSOID_BODY) {

        v3 radius = ((ellipsoid3d *)colBody->attributes)->radius;

        v3 points[3] = { (v3){ radius.x,0,0},
                         (v3){0, radius.y,0},
                         (v3){0,0, radius.z}};

        v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};
```

```
        for(int i = 0; i < 3; ++i) {
            v4 qpoint = (v4){ 0, points[i] };
            points[i] = (colBody->rotation * (qpoint,iq)).vec;
        }

        v3 max = {
            sqrt((points[0].x*points[0].x) +
                 (points[1].x*points[1].x) +
                 (points[2].x*points[2].x)),
            sqrt((points[0].y*points[0].y) +
                 (points[1].y*points[1].y) +
                 (points[2].y*points[2].y)),
            sqrt((points[0].z*points[0].z) +
                 (points[1].z*points[1].z) +
                 (points[2].z*points[2].z))
        };

        colBody->max = max;
        colBody->min = (v3){-max.x,-max.y,-max.z};

        return;

    } else if(colBody->shape & CYLINDER_BODY) {

        v3 points[2] = {
            (v3){0,0,-((cylinder3d *)colBody->attributes)->length/2},
            (v3){0,0, ((cylinder3d *)colBody->attributes)->length/2}
        };

        v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};
        for(int i = 0; i < 2; ++i) {
            v4 qpoint = (v4){ 0, points[i] };
            points[i] = (colBody->rotation * (qpoint * iq)).vec;
        }
        minMax(&colBody->min, &colBody->max, points, 2);

        // X calculation
        v3 a = colBody->min;
        v3 b = colBody->max;
```

```
        float r = ((cylinder3d *)colBody->attributes)->radius;
        float kx = sqrt(((a.y-b.y)*(a.y-b.y) +
                         (a.z-b.z)*(a.z-b.z)) /
                        ((a.x-b.x)*(a.x-b.x) +
                         (a.y-b.y)*(a.y-b.y) +
                         (a.z-b.z)*(a.z-b.z)));

        float ky = sqrt(((a.x-b.x)*(a.x-b.x) +
                         (a.z-b.z)*(a.z-b.z)) /
                        ((a.x-b.x)*(a.x-b.x) +
                         (a.y-b.y)*(a.y-b.y) +
                         (a.z-b.z)*(a.z-b.z)));

        float kz = sqrt(((a.x-b.x)*(a.x-b.x) +
                         (a.y-b.y)*(a.y-b.y)) /
                        ((a.x-b.x)*(a.x-b.x) +
                         (a.y-b.y)*(a.y-b.y) +
                         (a.z-b.z)*(a.z-b.z)));

        colBody->min.x -= kx*r;
        colBody->min.y -= ky*r;
        colBody->min.z -= kz*r;

        colBody->max.x += kx*r;
        colBody->max.y += ky*r;
        colBody->max.z += kz*r;

        return;

    } else if(colBody->shape & CAPSULE_BODY) {

        v3 points[2] = {
            (v3){0,0,-((capsule3d *)colBody->attributes)->length/2},
            (v3){0,0, ((capsule3d *)colBody->attributes)->length/2}
        };

        v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};
        for(int i = 0; i < 2; ++i) {
            v4 qpoint = (v4){ 0, points[i] };
```

```
            points[i] = (colBody->rotation * (qpoint * iq)).vec;
        }
        minMax(&colBody->min, &colBody->max, points, 2);
        colBody->min.x -= ((capsule3d *)colBody->attributes)->radius;
        colBody->min.y -= ((capsule3d *)colBody->attributes)->radius;
        colBody->min.z -= ((capsule3d *)colBody->attributes)->radius;

        colBody->max.x += ((capsule3d *)colBody->attributes)->radius;
        colBody->max.y += ((capsule3d *)colBody->attributes)->radius;
        colBody->max.z += ((capsule3d *)colBody->attributes)->radius;

        return;

    } else if(colBody->shape & CONVEX_HULL) {
        v3 points[colBody->shape.numVertices];

        v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};

        for(int i = 0; i < colBody->shape.numVertices; ++i) {
            points[i] = ((v3 *)colBody->attributes.vertices)[i];
            v4 qpoint = (v4){ 0, points[i] };
            points[i] = (colBody->rotation * (qpoint * iq)).vec;
        }

        minMax(&colBody->min,
               &colBody->max,
               points,
               colBody->shape.numVertices);

        return;

    }

    v3 points[8] = {(v3){min.x,min.y,min.z},
                    (v3){max.x,min.y,min.z},
                    (v3){min.x,max.y,min.z},
                    (v3){max.x,max.y,min.z},
                    (v3){min.x,min.y,max.z},
                    (v3){max.x,min.y,max.z},
```

```
                    (v3){min.x,max.y,max.z},
                    (v3){max.x,max.y,max.z}};

    v4 iq = (v4){colBody->rotation.w, invert(colBody->rotation.vec)};
    for(int i = 0; i < 8; ++i) {
        v4 qpoint = (v4){ 0, points[i] };
        points[i] = (colBody->rotation * (qpoint * iq)).vec;
    }

    minMax(&colBody->min, &colBody->max, points, 8);

}

// AABB Collision Function
bool col_AABB(colBody3d a, colBody3d b) {

    geometry::v3 minA = a.pos + a.min;
    geometry::v3 maxA = a.pos + a.max;

    geometry::v3 minB = b.pos + b.min;
    geometry::v3 maxB = b.pos + b.max;

    return !(minA.x > maxB.x  ||
             minA.y > maxB.y  ||
             minA.z > maxB.z  ||
             maxA.x <= minB.x ||
             maxA.y <= minB.y ||
             maxA.z <= minB.z);

}

// General Collision Simulation Function
void collisionDetection(simulation *sim) {

    colBody3d *a = sim->bodies3d[0];
    a->insideAABB = a->intersect = false;

    for(int i = 1; i < sim->bodyCount3d(); ++i) {
        colBody3d *b = sim->bodies3d[i];
```

```
        b->insideAABB = b->intersect = false;
    }

    for(int i = 1; i < sim->bodyCount3d(); ++i) {
        colBody3d *b = sim->bodies3d[i];
        b->insideAABB = b->intersect = false;
    }

    for(int i = 0; i < sim->bodyCount3d(); ++i) {
        colBody3d *a = sim->bodies3d[i];
        a->insideAABB = false;
        a->intersect = false;
    }

    for(int i = 0; i < sim->bodyCount3d()-1; ++i) {
        for(int j = i+1; j < sim->bodyCount3d(); ++j) {

            colBody3d *a = sim->bodies3d[i];
            colBody3d *b = sim->bodies3d[j];

            if(a->id != b->id) {
                if(col_AABB(*a, *b)) {

                    a->insideAABB = true;
                    b->insideAABB = true;

                    bool newIntersect = col_GJK(*a, *b);
                    a->intersect |= newIntersect;
                    b->intersect |= newIntersect;

                }
            }
        }
    }
}
```

A.7 Função *NearestSimplex*

```
bool nearestSimplexGJK(v3 *shape, int *numPoints, v3 *direction) {

    switch(*numPoints) {
        case 2: {

            // A = 1, B = 0

            v3 A = shape[1];
            v3 B = shape[0];

            v3 ab = B - A;
            v3 a0 = (v3){0,0,0} - A;

            if(dot(ab,a0) > 0) {
                // simplex = [A,B]
                shape[0] = A;
                shape[1] = B;
                *direction = cross(cross(ab, a0), ab);
                // Dir = ab X a0 X ab
            } else {
                shape[0] = A;
                *numPoints -= 1; // simplex = [A]
                *direction = a0; // Dir = a0
            }
        } break;
        case 3: {

            v3 A = shape[2];
            v3 B = shape[1];
            v3 C = shape[0];

            v3 ab = B - A;
            v3 ac = C - A;
            v3 abc = cross(ab, ac); // ab X ac

            v3 a0 = (v3){0,0,0} - A;

            if(dot(cross(abc, ac), a0) > 0) {
```

```
            if(dot(ac, a0) > 0) {
                shape[0] = A;
                shape[1] = C;
                *numPoints -= 1; // simplex = [A,C]
                *direction = cross(cross(ac,a0), ac);
                // Dir = ac X a0 X ac
            } else {
                if(dot(ab, a0) > 0) {
                    shape[0] = A;
                    shape[1] = B;
                    *numPoints -= 1; // simplex = [A,B]
                    *direction = cross(cross(ab,a0), ab);
                    // Dir = ab X a0 X ab
                } else {
                    shape[0] = A;
                    *numPoints -= 2; // simplex = [A]
                    *direction = a0; // Dir = a0
                }
            }
        } else {
            if(dot(cross(ab,abc), a0) > 0) {
                if(dot(ab, a0) > 0) {
                    shape[0] = A;
                    shape[1] = B;
                    *numPoints -= 1; // simplex = [A,B]
                    *direction = cross(cross(ab,a0), ab);
                    // Dir = ab X a0 X ab
                } else {
                    shape[0] = A;
                    *numPoints -= 2; // simplex = [A]
                    *direction = a0; // Dir = a0
                }
            } else {
                if(dot(abc, a0) > 0) {
                    shape[0] = A;
                    shape[1] = B;
                    shape[2] = C;
                    // simplex = [A,B,C]
                    *direction = abc; // Dir = abc
```

```
                } else {
                    shape[0] = A;
                    shape[1] = C;
                    shape[2] = B;
                    // simplex = [A,C,B]
                    *direction = invert(abc); // Dir = -abc
                }
            }
        }
    } break;
    case 4: {

        v3 A = shape[3];
        v3 B = shape[2];
        v3 C = shape[1];
        v3 D = shape[0];

        v3 ab = B - A;
        v3 ac = C - A;
        v3 ad = D - A;

        v3 a0 = (v3){0,0,0} - shape[3];

        v3 abc = cross(ab, ac);
        v3 acd = cross(ac, ad);
        v3 adb = cross(ad, ab);

        // ABC
        if(dot(abc, a0) < 0) {
            if(dot(cross(abc, ac), a0) > 0) {
                if(dot(ac, a0) > 0) {
                    shape[0] = A;
                    shape[1] = C;
                    *numPoints -= 2; // simplex = [A,C]
                    *direction = cross(cross(ac,a0), ac);
                    // Dir = ac X a0 X ac
                } else {
                    if(dot(ab, a0) > 0) {
                        shape[0] = A;
```

```
                    shape[1] = B;
                    *numPoints -= 2; // simplex = [A,B]
                    *direction = cross(cross(ab,a0), ab);
                    // Dir = ab X a0 X ab
                } else {
                    shape[0] = A;
                    *numPoints -= 3; // simplex = [A]
                    *direction = a0; // Dir = a0
                }
            }
        } else {
            if(dot(cross(ab,abc), a0) > 0) {
                if(dot(ab, a0) > 0) {
                    shape[0] = A;
                    shape[1] = B;
                    *numPoints -= 2; // simplex = [A,B]
                    *direction = cross(cross(ab,a0), ab);
                    // Dir = ab X a0 X ab
                } else {
                    shape[0] = A;
                    *numPoints -= 3; // simplex = [A]
                    *direction = a0; // Dir = a0
                }
            } else {
                shape[0] = A;
                shape[1] = C;
                shape[2] = B;
                *numPoints -= 1;
                // simplex = [A,C,B]
                *direction = invert(abc); // Dir = -abc
            }
        }
    }

    // ACD
    else if(dot(acd, a0) < 0) {
        if(dot(cross(acd, ad), a0) > 0) {
            if(dot(ad, a0) > 0) {
                shape[0] = A;
```

```
            shape[1] = D;
            *numPoints -= 2; // simplex = [A,D]
            *direction = cross(cross(ad,a0), ad);
            // Dir = ad X a0 X ad
        } else {
            if(dot(ac, a0) > 0) {
                shape[0] = A;
                shape[1] = C;
                *numPoints -= 2; // simplex = [A,C]
                *direction = cross(cross(ac,a0), ac);
                // Dir = ac X a0 X ac
            } else {
                shape[0] = A;
                *numPoints -= 3; // simplex = [A]
                *direction = a0; // Dir = a0
            }
        }
    } else {
        if(dot(cross(ac,acd), a0) > 0) {
            if(dot(ac, a0) > 0) {
                shape[0] = A;
                shape[1] = C;
                *numPoints -= 2; // simplex = [A,C]
                *direction = cross(cross(ac,a0), ac);
                // Dir = ac X a0 X ac
            } else {
                shape[0] = A;
                *numPoints -= 3; // simplex = [A]
                *direction = a0; // Dir = a0
            }
        } else {
            shape[0] = A;
            shape[1] = D;
            shape[2] = C;
            *numPoints -= 1;
            // simplex = [A,D,C]
            *direction = invert(acd); // Dir = -acd
        }
    }
```

```
}

// ADB
else if(dot(adb, a0) < 0) {
    if(dot(cross(adb, ab), a0) > 0) {
        if(dot(ab, a0) > 0) {
            shape[0] = A;
            shape[1] = B;
            *numPoints -= 2; // simplex = [A,B]
            *direction = cross(cross(ab,a0), ab);
            // Dir = ab X a0 X ab
        } else {
            if(dot(ad, a0) > 0) {
                shape[0] = A;
                shape[1] = D;
                *numPoints -= 2; // simplex = [A,D]
                *direction = cross(cross(ad,a0), ad);
                // Dir = ad X a0 X ad
            } else {
                shape[0] = A;
                *numPoints -= 3; // simplex = [A]
                *direction = a0; // Dir = a0
            }
        }
    } else {
        if(dot(cross(ad,adb), a0) > 0) {
            if(dot(ad, a0) > 0) {
                shape[0] = A;
                shape[1] = D;
                *numPoints -= 2; // simplex = [A,D]
                *direction = cross(cross(ad,a0), ad);
                // Dir = ad X a0 X ad
            } else {
                shape[0] = A;
                *numPoints -= 3; // simplex = [A]
                *direction = a0; // Dir = a0
            }
        } else {
            shape[0] = A;
```

```
                        shape[1] = B;
                        shape[2] = D;
                        *numPoints -= 1;
                        // simplex = [A,B,D]
                        *direction = invert(adb); // Dir = -adb
                    }
                }
            } else {
                return true;
            }
        } break;
    }
    return false;
}
```

A.8 Função GJK

```
bool col_GJK(colBody3d a, colBody3d b) {
    v3 direction = (v3){1,0,0}; // start direction
    int numPoints = 0; // initial size of simplex array
    v3 simplex[4]; /* simplex is not required
                      to be any larger than 4 points in 3d */

    v3 A = a.support(a, direction) -
           b.support(b, invert(direction));

    if(dot(A, direction) < 0.f) {
        return false;
    }

    simplex[numPoints++] = A;
    v3 D = invert(A);

    while(true) {

        A = a.support(a, D) - b.support(b, invert(D));

       if(dot(A, D) < 0.f) {
           return false;
```

```
        }

        simplex[numPoints++] = A;
        bool containsOrigin = nearestSimplexGJK(simplex, &numPoints, &D);
        if(containsOrigin) {
            return true;
        }
    }
}
```